CSC3100: Graph II

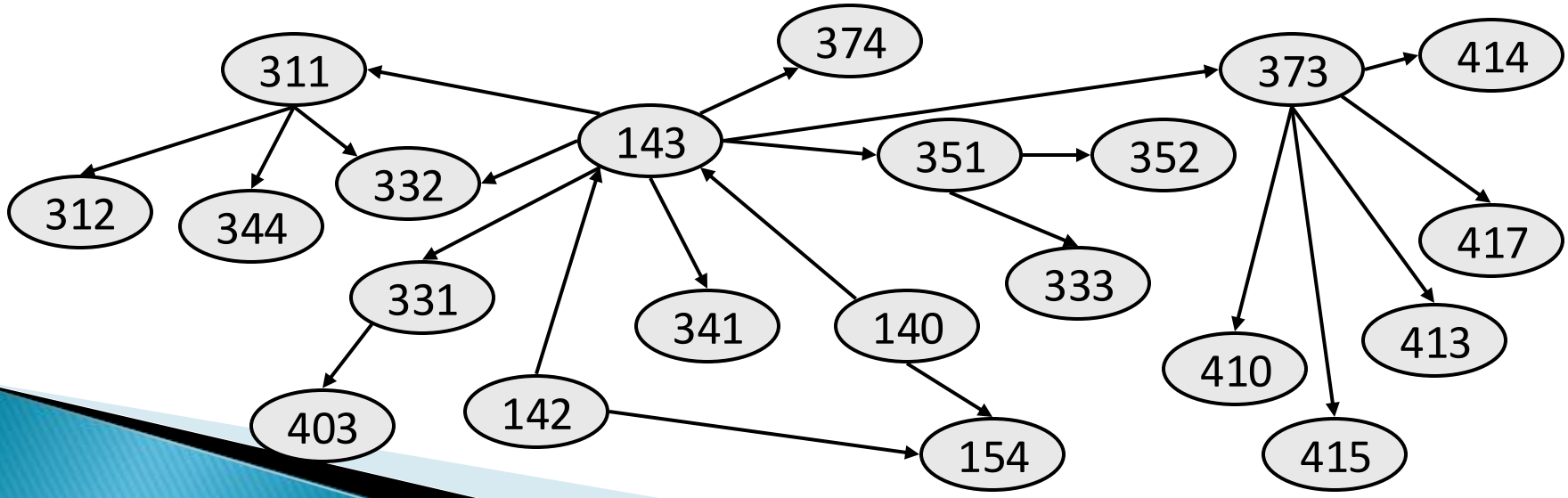
Tianshu Yu

2024 Fall

Topological Sorting

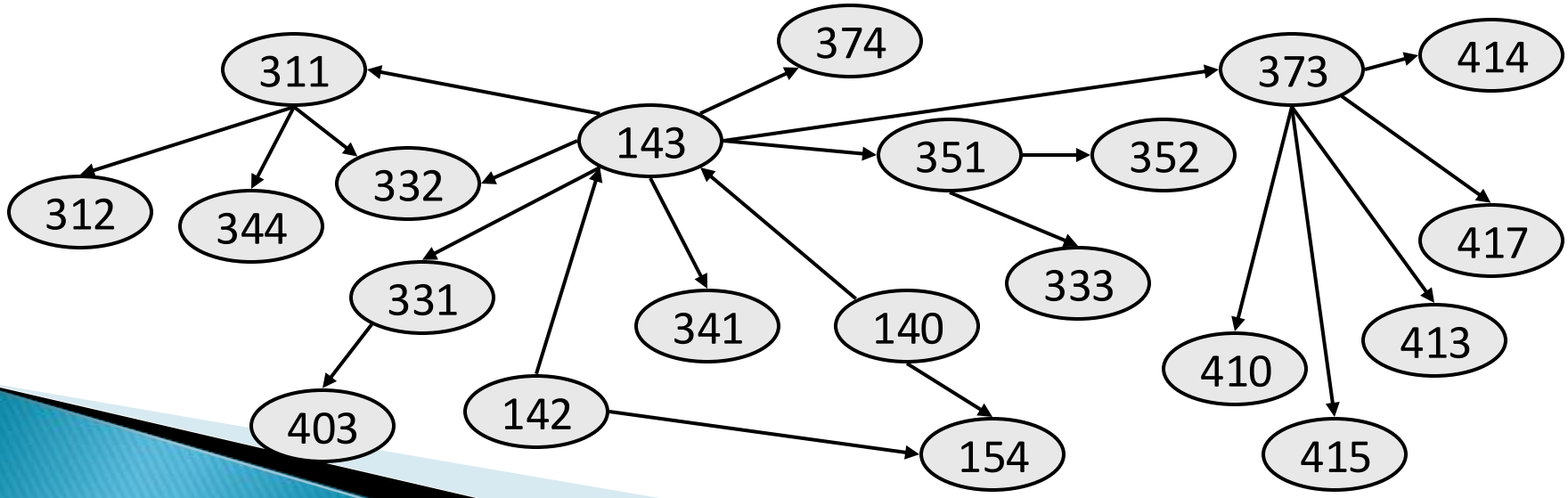
Ordering a graph

- Suppose we have a directed acyclic graph (DAG) of courses, and we want to find an order in which the courses can be taken.
  - Must take all prereqs before you can take a given course. Example:
    - [142, 143, 140, 154, 341, 374, 331, 403, 311, 332, 344, 312, 351, 333, 352, 373, 414, 410, 417, 413, 415]
    - There might be more than one allowable ordering.
  - How can we find a valid ordering of the vertices?



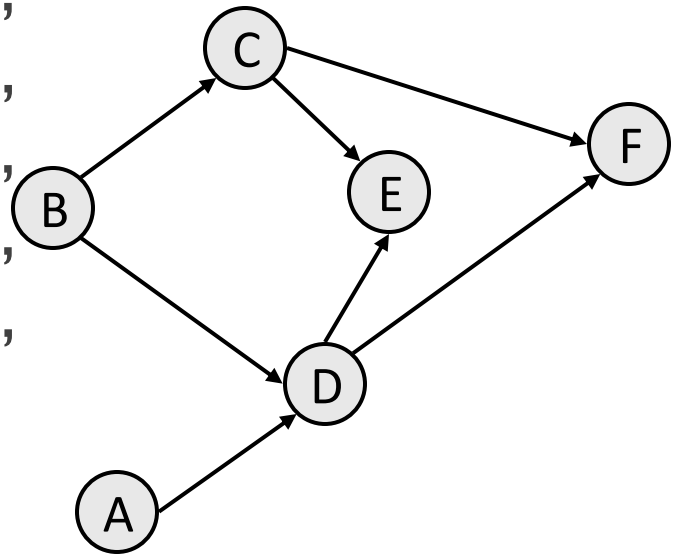
Topological Sort

- **Topological sort:** Given a digraph $G = (V, E)$, a total ordering of G's vertices such that for every edge (v, w) in E, vertex v precedes w in the ordering. Examples:
  - determining the order to recalculate updated cells in a spreadsheet
  - finding an order to recompile files that have dependencies
    - (any problem of finding an order to perform tasks with dependencies)



Topo sort example

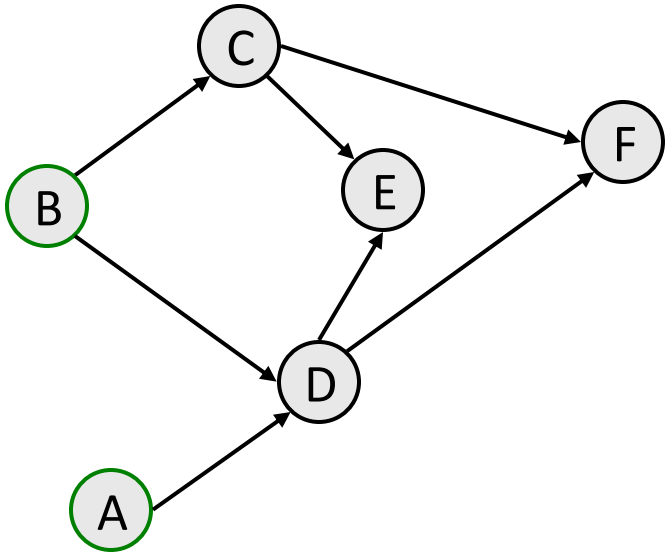
- How many valid topological sort orderings can you find for the vertices in the graph below?
  - [A, B, C, D, E, F], [A, B, C, D, F, E],
  - [A, B, D, C, E, F], [A, B, D, C, F, E],
  - [B, A, C, D, E, F], [B, A, C, D, F, E],
  - [B, A, D, C, E, F], [B, A, D, C, F, E],
  - [B, C, A, D, E, F], [B, C, A, D, F, E],
  - ...



  - What if there were a new vertex G unconnected to the others?

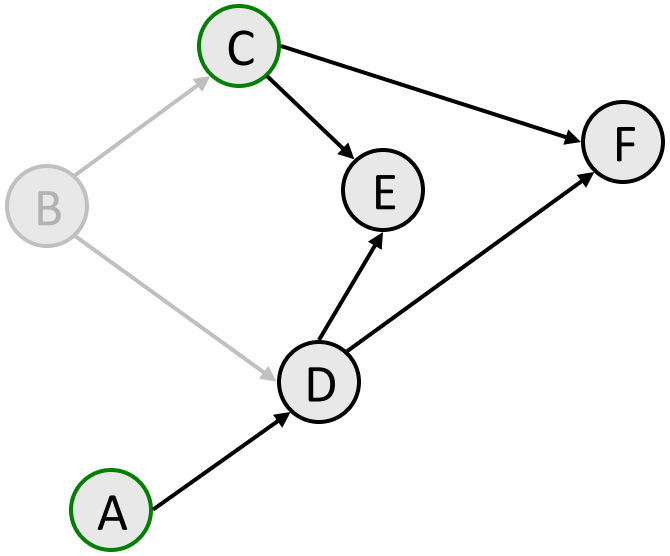
Topo sort: Algorithm 1

- function topologicalSort():
  - *ordering* := { }.
  - Repeat until graph is empty:
    - Find a vertex *v* with in-degree of 0 (no incoming edges).
      - (If there is no such vertex, the graph cannot be sorted; stop.)
    - Delete *v* and all of its outgoing edges from the graph.
    - *ordering* += *v* .



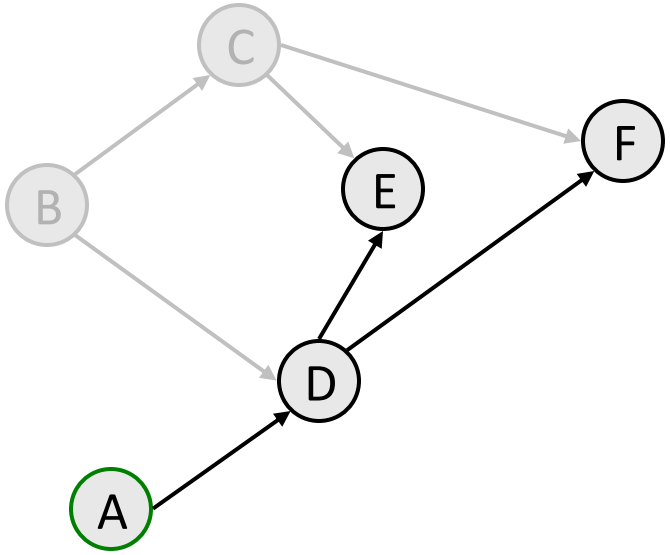
Topo sort example

- function topologicalSort():
  - *ordering* := { }.
  - Repeat until graph is empty:
    - Find a vertex *v* with in-degree of 0 (no incoming edges).
      - (If there is no such vertex, the graph cannot be sorted; stop.)
    - Delete *v* and all of its outgoing edges from the graph.
    - *ordering* += *v* .
  - *ordering* = { B }



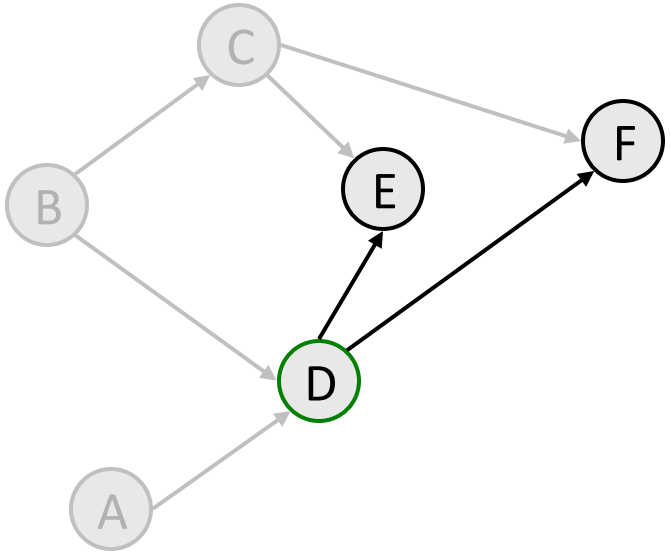
Topo sort example

- function topologicalSort():
  - *ordering* := { }.
  - Repeat until graph is empty:
    - Find a vertex *v* with in-degree of 0 (no incoming edges).
      - (If there is no such vertex, the graph cannot be sorted; stop.)
    - Delete *v* and all of its outgoing edges from the graph.
    - *ordering* += *v* .
  - *ordering* = { B, C }



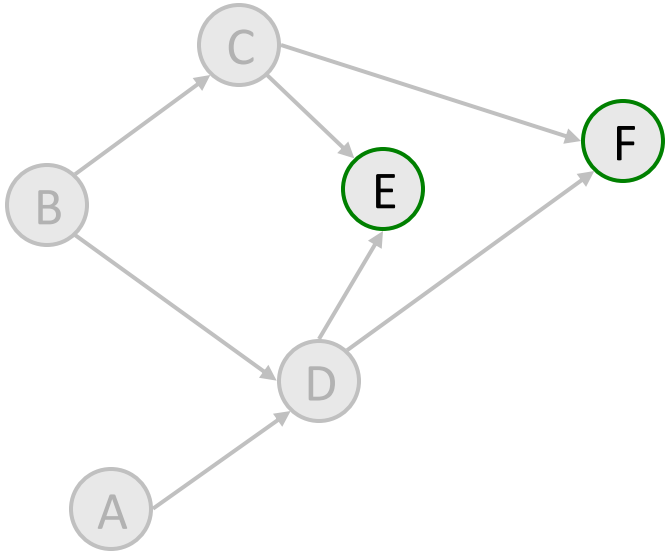
Topo sort example

- function topologicalSort():
  - *ordering* := { }.
  - Repeat until graph is empty:
    - Find a vertex *v* with in-degree of 0 (no incoming edges).
      - (If there is no such vertex, the graph cannot be sorted; stop.)
    - Delete *v* and all of its outgoing edges from the graph.
    - *ordering* += *v* .
  - *ordering* = { B, C, A }



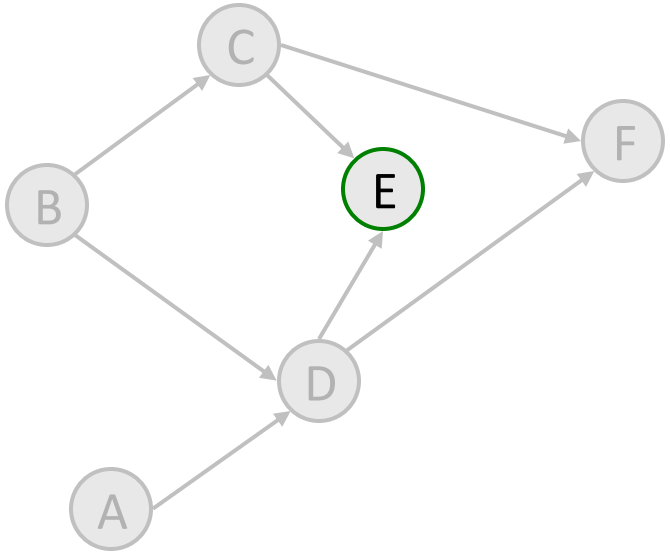
Topo sort example

- function topologicalSort():
  - *ordering* := { }.
  - Repeat until graph is empty:
    - Find a vertex *v* with in-degree of 0 (no incoming edges).
      - (If there is no such vertex, the graph cannot be sorted; stop.)
    - Delete *v* and all of its outgoing edges from the graph.
    - *ordering* += *v* .
  - *ordering* = { B, C, A, D }



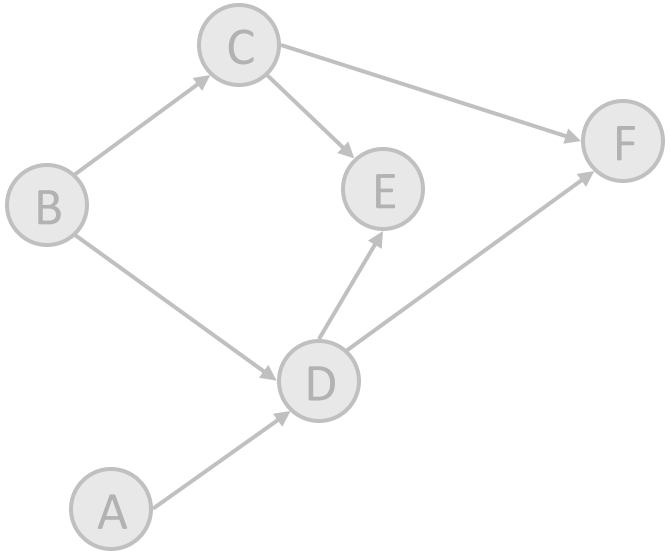
Topo sort example

- function topologicalSort():
  - *ordering* := { }.
  - Repeat until graph is empty:
    - Find a vertex *v* with in-degree of 0 (no incoming edges).
      - (If there is no such vertex, the graph cannot be sorted; stop.)
    - Delete *v* and all of its outgoing edges from the graph.
    - *ordering* += *v* .
  - *ordering* = { B, C, A, D, F }



Topo sort example

- function topologicalSort():
  - *ordering* := { }.
  - Repeat until graph is empty:
    - Find a vertex *v* with in-degree of 0 (no incoming edges).
      - (If there is no such vertex, the graph cannot be sorted; stop.)
    - Delete *v* and all of its outgoing edges from the graph.
    - *ordering* += *v* .
  - *ordering* = { B, C, A, D, F, E }

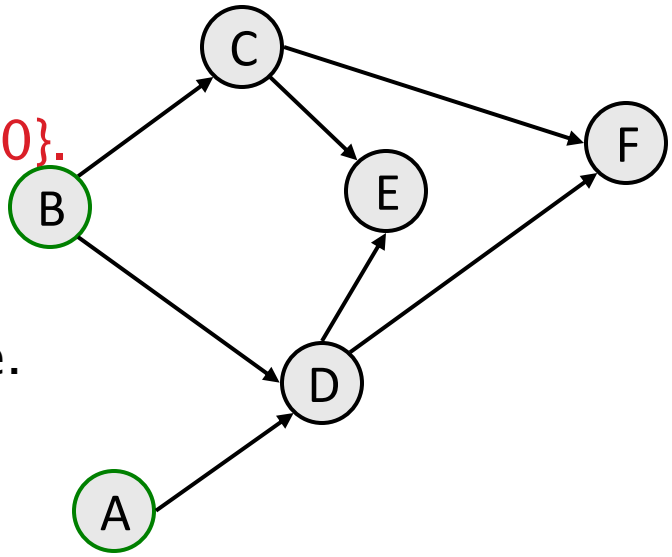


Revised algorithm

- We don't want to literally delete vertices and edges from the graph while trying to topological sort it; so let's revise the algorithm:
  - *map* := {each vertex → its in-degree}.
  - *queue* := {all vertices with in-degree = 0}.
  - *ordering* := { }.
  - Repeat until queue is empty:
    - Dequeue the first vertex *v* from the queue.
    - *ordering* += *v*.
    - Decrease the in-degree of all *v*'s neighbors by 1 in the *map*.
    - *queue* += {any neighbors whose in-degree is now 0}.
  - If all vertices are processed, success. Otherwise, there is a cycle.

Topo sort example 2

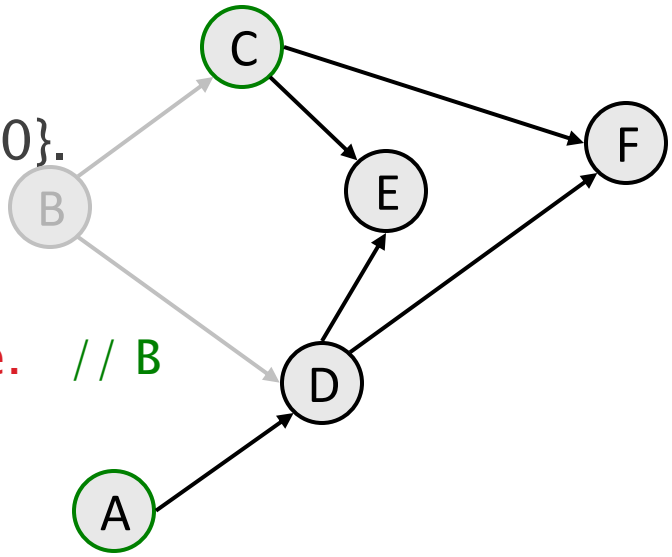
- function topologicalSort():
  - *map* := {each vertex → its in-degree}.
  - *queue* := {all vertices with in-degree = 0}.
  - *ordering* := { }.
  - Repeat until queue is empty:
    - Dequeue the first vertex *v* from the queue.
    - *ordering* += *v*.
    - Decrease the in-degree of all *v*'s neighbors by 1 in the *map*.
    - *queue* += {any neighbors whose in-degree is now 0}.



  - map := { A=0, B=0, C=1, D=2, E=2, F=2 }
  - queue := { B, A }
  - ordering := { }

Topo sort example 2

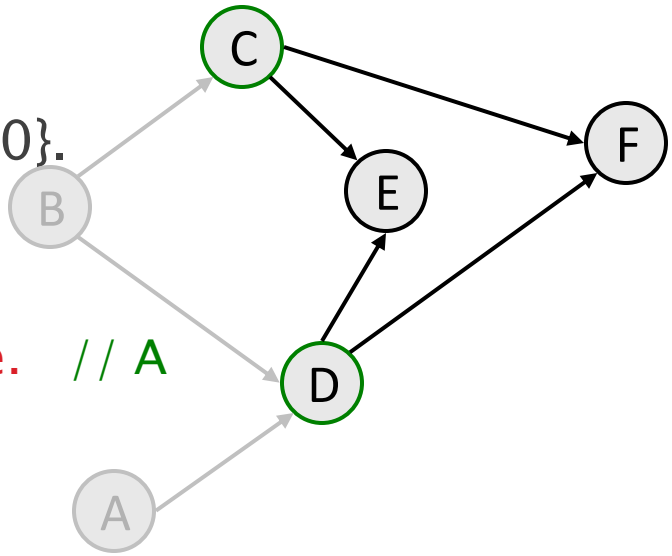
- function topologicalSort():
  - *map* := {each vertex → its in-degree}.
  - *queue* := {all vertices with in-degree = 0}.
  - *ordering* := { }.
  - Repeat until queue is empty:
    - Dequeue the first vertex *v* from the queue. // B
    - *ordering* += *v*.
    - Decrease the in-degree of all *v*'s neighbors by 1 in the *map*. // C, D
    - *queue* += {any neighbors whose in-degree is now 0}.



  - map := { A=0, B=0, **C=0**, **D=1**, E=2, F=2 }
  - queue := { A, **C** }
  - ordering := { **B** }

Topo sort example 2

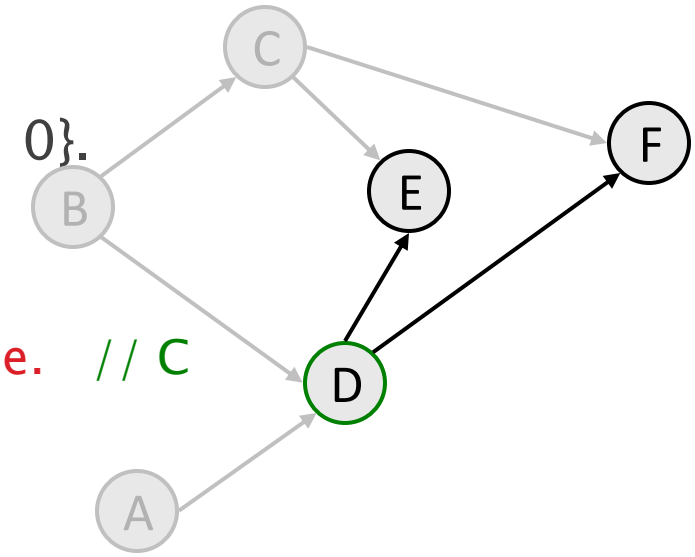
- function topologicalSort():
  - *map* := {each vertex → its in-degree}.
  - *queue* := {all vertices with in-degree = 0}.
  - *ordering* := { }.
  - Repeat until queue is empty:
    - Dequeue the first vertex *v* from the queue. // A
    - *ordering* += *v*.
    - Decrease the in-degree of all *v*'s neighbors by 1 in the *map*. // D
    - *queue* += {any neighbors whose in-degree is now 0}.



  - map := { A=0, B=0, C=0, **D=0**, E=2, F=2 }
  - queue := { C, **D** }
  - ordering := { B, **A** }

Topo sort example 2

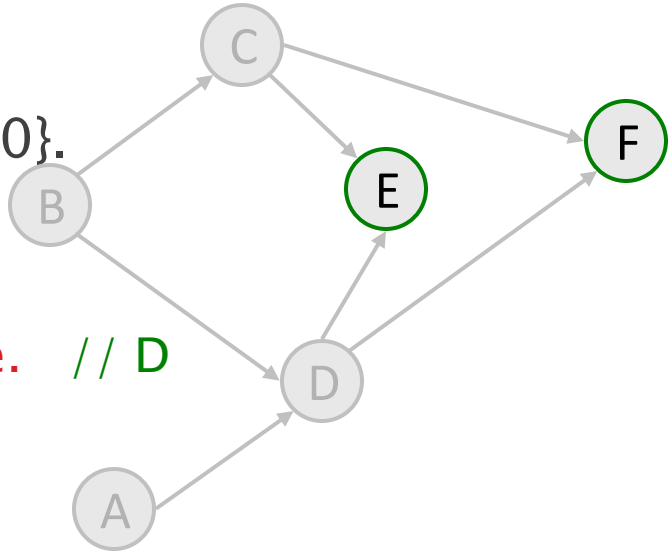
- function topologicalSort():
  - *map* := {each vertex → its in-degree}.
  - *queue* := {all vertices with in-degree = 0}.
  - *ordering* := { }.
  - Repeat until queue is empty:
    - Dequeue the first vertex *v* from the queue. // C
    - *ordering* += *v*.
    - Decrease the in-degree of all *v*'s neighbors by 1 in the *map*. // E, F
    - *queue* += {any neighbors whose in-degree is now 0}.



  - map := { A=0, B=0, C=0, D=0, E=1, F=1 }
  - queue := { D }
  - ordering := { B, A, C }

Topo sort example 2

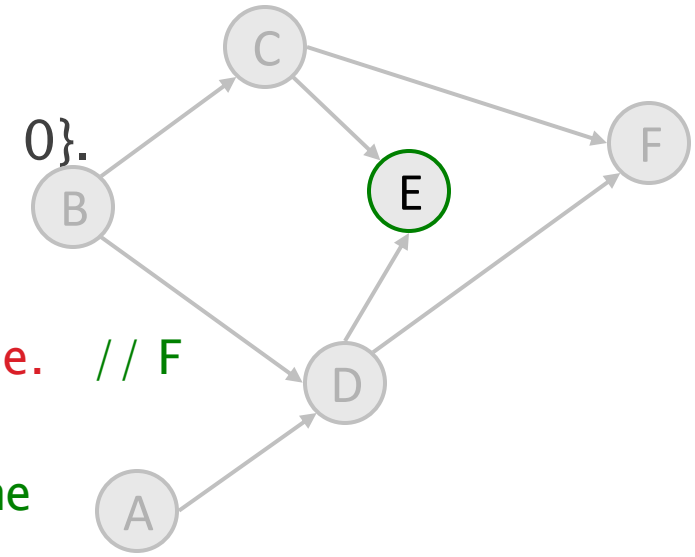
- function topologicalSort():
  - *map* := {each vertex → its in-degree}.
  - *queue* := {all vertices with in-degree = 0}.
  - *ordering* := { }.
  - Repeat until queue is empty:
    - Dequeue the first vertex *v* from the queue. // D
    - *ordering* += *v*.
    - Decrease the in-degree of all *v*'s neighbors by 1 in the *map*. // F, E
    - *queue* += {any neighbors whose in-degree is now 0}.



  - map := { A=0, B=0, C=0, D=0, E=**0**, F=**0** }
  - queue := { **F**, **E** }
  - ordering := { B, A, C, **D** }

Topo sort example 2

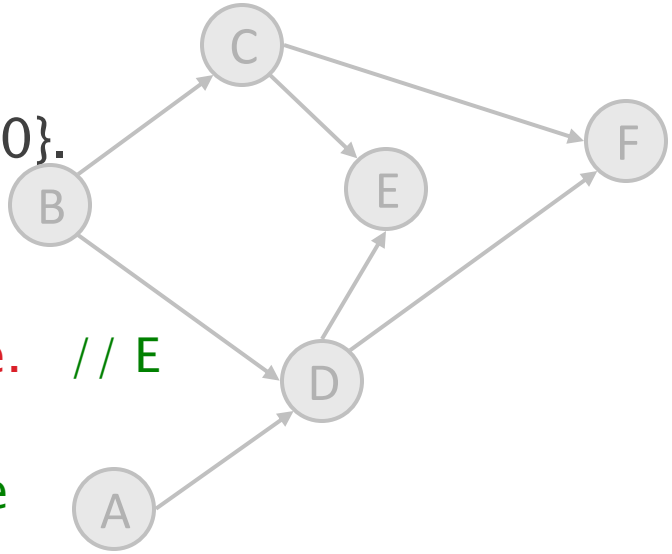
- function topologicalSort():
  - *map* := {each vertex → its in-degree}.
  - *queue* := {all vertices with in-degree = 0}.
  - *ordering* := { }.
  - Repeat until queue is empty:
    - Dequeue the first vertex *v* from the queue. // F
    - *ordering* += *v*.
    - Decrease the in-degree of all *v*'s neighbors by 1 in the *map*. // none
    - *queue* += {any neighbors whose in-degree is now 0}.



  - map := { A=0, B=0, C=0, D=0, E=0, F=0 }
  - queue := { E }
  - ordering := { B, A, C, D, **F** }

Topo sort example 2

- function topologicalSort():
  - *map* := {each vertex → its in-degree}.
  - *queue* := {all vertices with in-degree = 0}.
  - *ordering* := { }.
  - Repeat until queue is empty:
    - Dequeue the first vertex *v* from the queue. // E
    - *ordering* += *v*.
    - Decrease the in-degree of all *v*'s neighbors by 1 in the *map*. // none
    - *queue* += {any neighbors whose in-degree is now 0}.



  - map := { A=0, B=0, C=0, D=0, E=0, F=0 }
  - queue := { }
  - ordering := { B, A, C, D, F, E }

Topo sort runtime

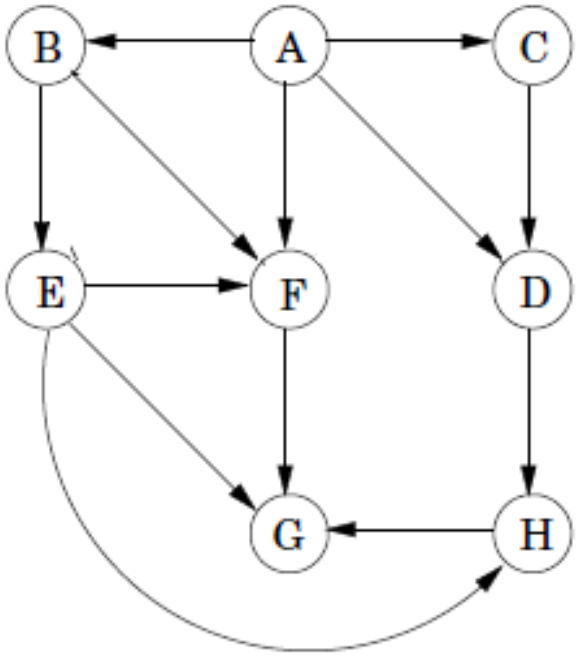
- What is the runtime of our topological sort algorithm?
  - (with an "adjacency map" graph internal representation)

  - function topologicalSort():
    - *map* := {each vertex → its in-degree}. // O(*V*)
    - *queue* := {all vertices with in-degree = 0}.
    - *ordering* := { }.
    - Repeat until queue is empty: // O(*V*)
      - Dequeue the first vertex *v* from the queue. // O(*1*)
      - *ordering* += *v*. // O(*1*)
      - Decrease the in-degree of all *v*'s neighbors by 1 in the *map*. // O(*E*) for all passes
      - *queue* += {any neighbors whose in-degree is now 0}.

  - Overall: O(*V* + *E*) ; essentially O(*V*) time on a sparse graph (fast!)

Exercise

- Compute the topological sort for the following graph



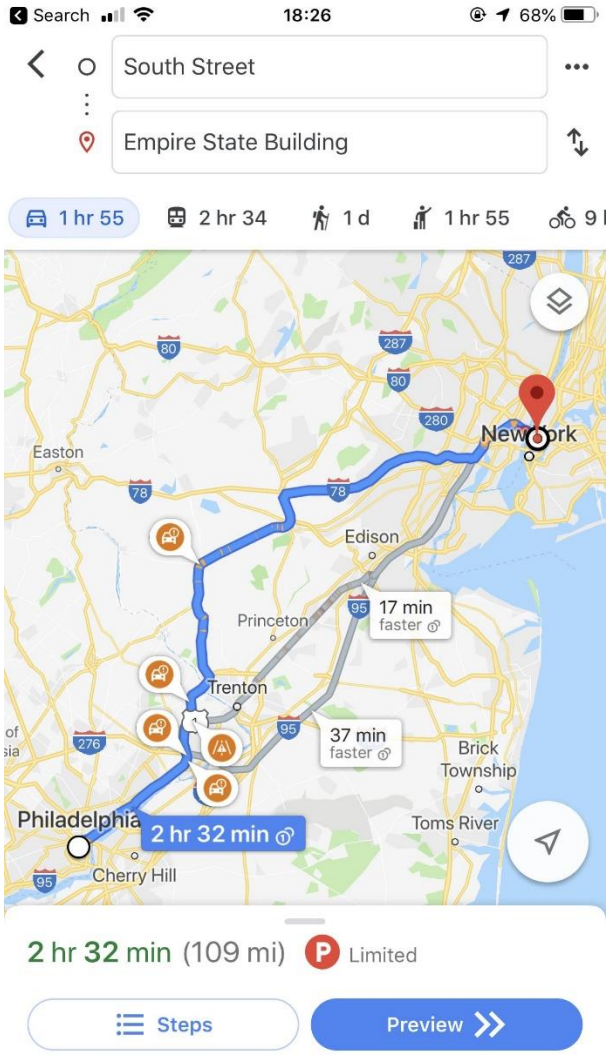
Shortest Path

Outline

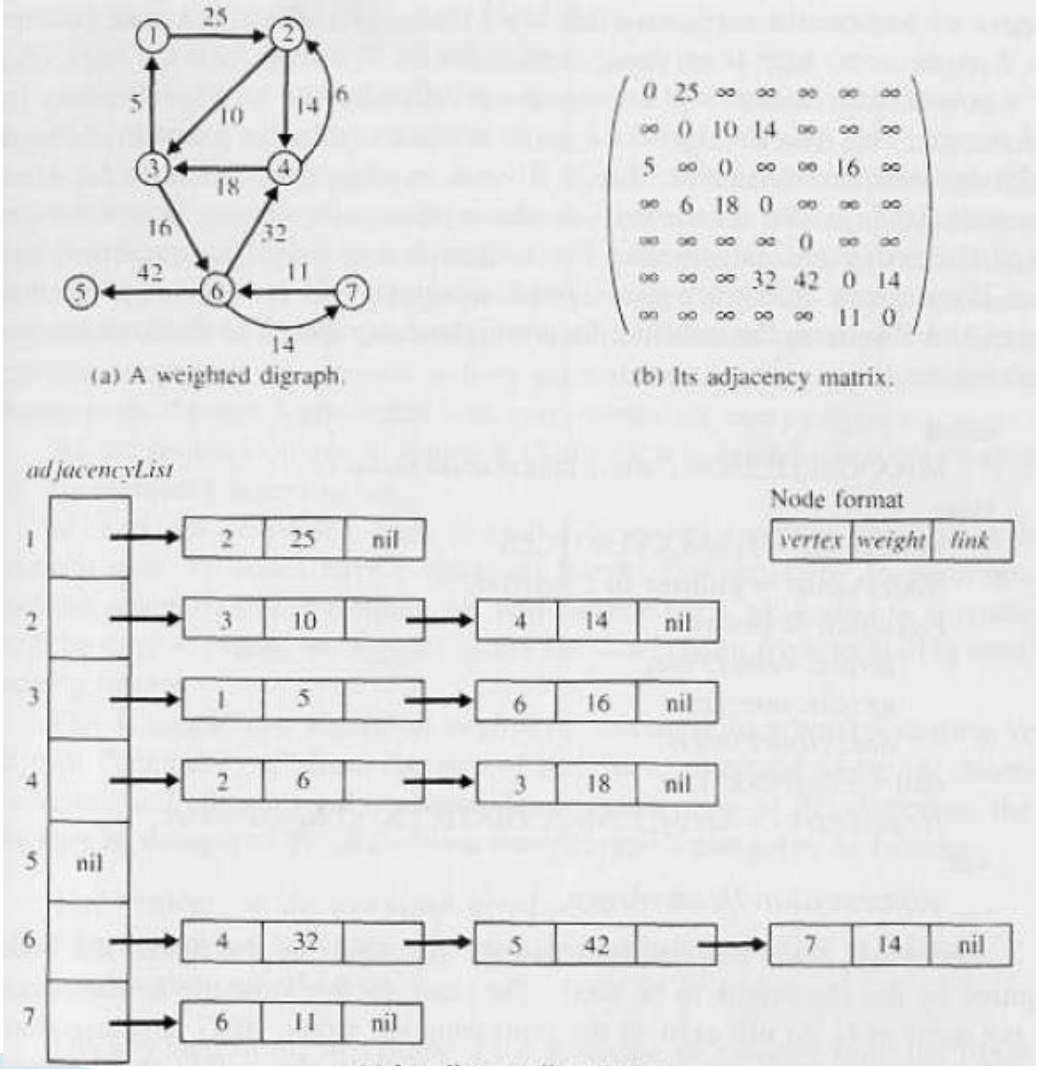
- We focus on weighted graphs
- Graphs with non-negative weights
  - Single-Source Shortest Path: Dijkstra's algorithm
- All-Pair Shortest Path: Floyd's algorithm
- Graphs with negative weights
  - Bellman-Ford algorithm

Weighted graphs

- In real world graphs, each edge may have weights
  - On road networks, each edge (a road segment) have a weight, which may be the distance between two road junctions or the travel time from one junction to another
  - In navigation systems, e.g., Google Map, we may want to find the path with minimum travel time between two locations



Weighted Graph Representation



Shortest path problems

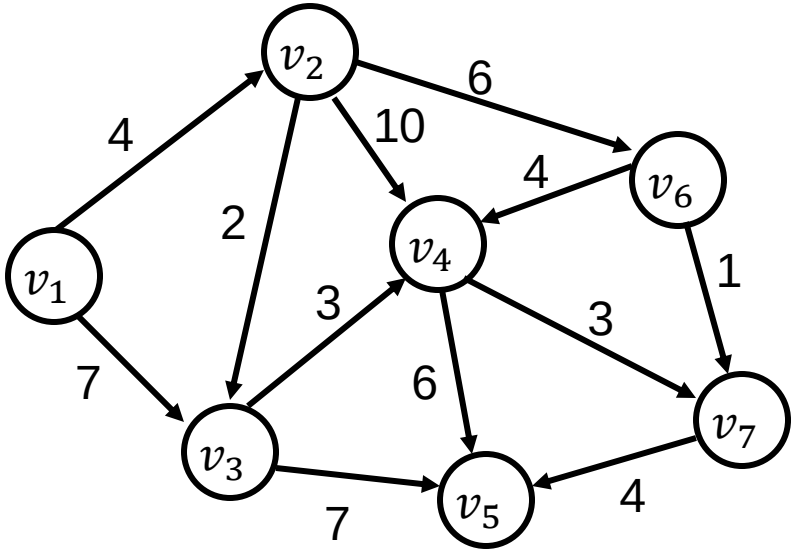
- How can we find the shortest route between two points on a road map?

- Model the problem as a graph problem:
  - Road map is a weighted graph:
    vertices = cities
    edges = road segments between cities
    edge weights = road distances

  - Goal: find a shortest path between two vertices (cities)

An example of weighted graph

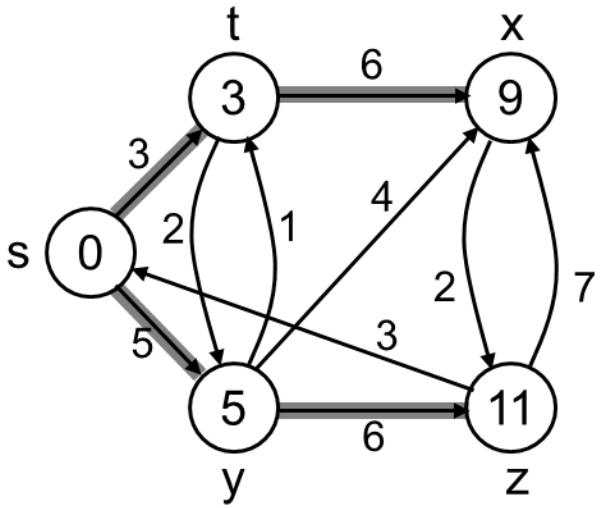
- A weighted graph is a graph such that each edge e is associated with a weight $w(e)$
  - We focus on directed graphs
  - The solution can be easily extended to undirected graphs



$w(v_1, v_2) = 4$
$w(v_1, v_3) = 7$
…

Shortest path problems

- **Input:**
  - Directed graph G = (V, E)
  - Weight function w : E → **R**



- **Weight of path** $p = \langle v_0, v_1, \ldots, v_k \rangle$

$$w(p) = \sum_{i=1}^{k} w(v_{i-1}, v_i)$$

- **Shortest-path weight** from u to v:

$$\delta(u, v) = \min_{p} \begin{cases} w(p) : u \overset{p}{\rightsquigarrow} v & \text{if there exists a path from u to v} \\ \infty & \text{otherwise} \end{cases}$$

- **Note:** there might be multiple shortest paths from u to v

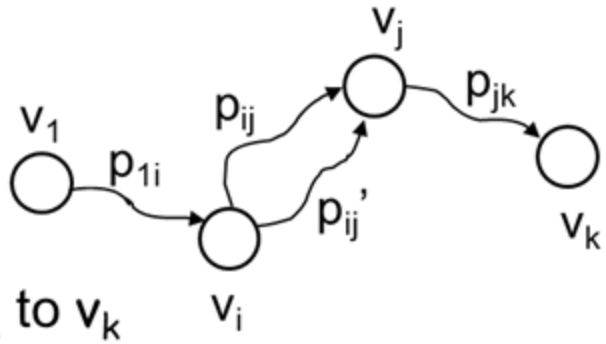
Variants of shortest path

- **Single-source shortest paths**
  - $G = (V, E) \Rightarrow$ find a shortest path from a given source vertex s to each vertex $v \in V$
- **Single-destination shortest paths**
  - Find a shortest path to a given destination vertex **t** from each vertex v
  - Reversing the direction of each edge $\Rightarrow$ single-source
- **Single-pair shortest path**
  - Find a shortest path from u to v for given vertices u and v
- **All-pairs shortest-paths**
  - Find a shortest path from u to v for every pair of vertices u and v

Optimal substructure theorem

Given:

- A weighted, directed graph G = (V, E)
- A weight function w: E → **R**,
- A shortest path $p = \langle v_1, v_2, \ldots, v_k \rangle$ from v_1 to v_k
- A subpath of p: $p_{ij} = \langle v_i, v_{i+1}, \ldots, v_j \rangle$, with $1 \le i \le j \le k$



Then: p_{ij} is a shortest path from v_i to v_j

Proof: $p = v_1 \overset{p_{1i}}{\rightsquigarrow} v_i \overset{p_{ij}}{\rightsquigarrow} v_j \overset{p_{jk}}{\rightsquigarrow} v_k$

$$w(p) = w(p_{1i}) + w(p_{ij}) + w(p_{jk})$$

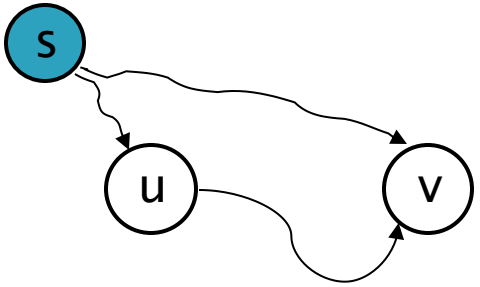
Assume $\exists\ p_{ij}'$ from v_i to v_j with $w(p_{ij}') < w(p_{ij})$

$\Rightarrow w(p') = w(p_{1i}) + w(p_{ij}') + w(p_{jk}) < w(p)$ contradiction!

Triangle inequality

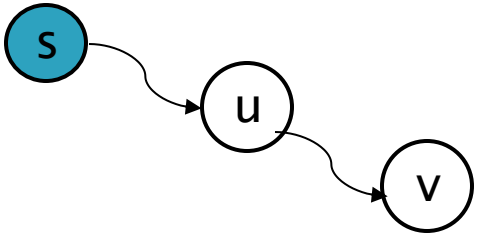
- For all $(u, v) \in E$, we have:

  $\delta(s, v) \leq \delta(s, u) + \delta(u, v)$



Proof?

- If u is on the shortest path to v we have the equality sign



Cycles

- Can shortest paths contain cycles?

- Negative-weight cycles No!
  - Shortest path is not well defined

- Positive-weight cycles: No!
  - By removing the cycle, we can get a shorter path

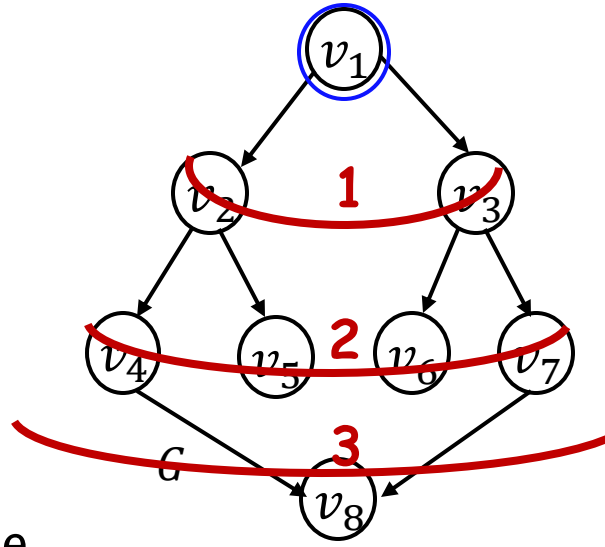
Shortest path: unweighted graph

- A simple case: unweighted graph
  - How to find the shortest path? Use BFS!

- A simple algorithm

  1. Mark the starting vertex, s
  2. Find and mark all unmarked vertices adjacent to s
  3. Find and mark all unmarked vertices adjacent to the marked vertices
  4. Repeat Step 3 until all vertices are marked

- For each vertex, keep track of
  - whether the adjacent vertex has been marked
  - its distance from $s(d_v)$
  - previous vertex of the path from $s(p_v)$



Shortest path: unweighted graph

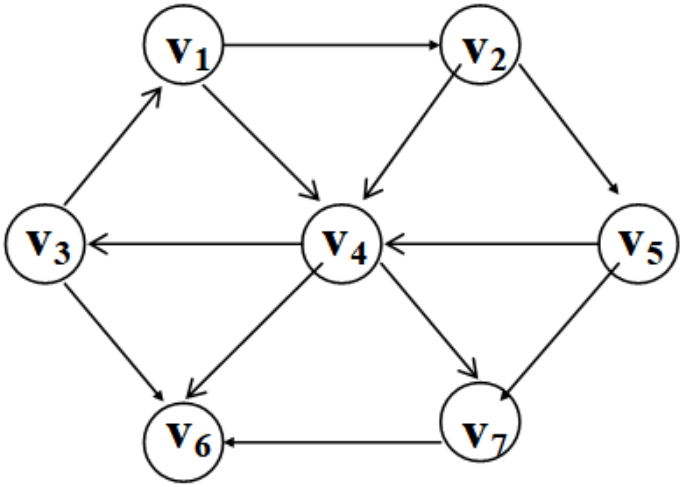
/* Pseudocode for unweighted shortest-path algorithm with O(|E| + |V|) time*/

```
void unweighted(Vertex s) {
      Queue<Vertex> q = new Queue<Vertex>();
      for each vertex { v.dist = INFINITY;}
      d_s = 0;
      q.enqueue(s);
      while(!q.isEmpty()){
            Vertex v = q.dequeue();
       for each Vertex w adjacent to v
              if(d_w == INFINITY){
                        d_w = d_v + 1;
                        p_w = v;
                        q.enqueue(w);
                }
            }
    }
}
```

Running time is O(|E| + |V|)

Shortest path: unweighted graph

Start vertex

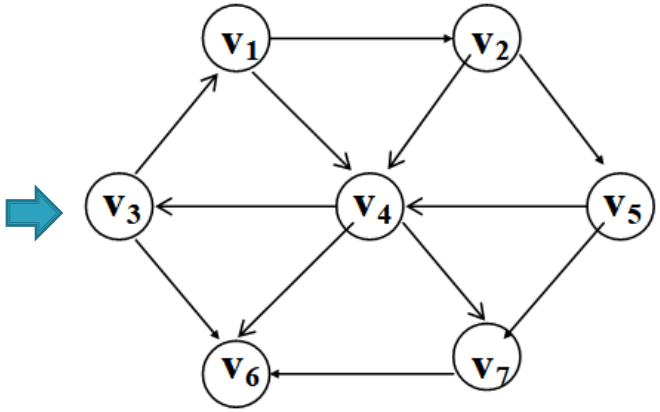


v	*Known*	d_v	p_v
v_1	F	∞	0
v_2	F	∞	0
v_3	F	0	0
v_4	F	∞	0
v_5	F	∞	0
v_6	F	∞	0
v_7	F	∞	0

Shortest path: unweighted graph

	Initial State		
v	*Known*	d_v	p_v
v_1	F	∞	0
v_2	F	∞	0
v_3	F	0	0
v_4	F	∞	0
v_5	F	∞	0
v_6	F	∞	0
v_7	F	∞	0
Q	v_3		

	v_3 Dequeued		
v	*Known*	d_v	p_v
v_1	F	1	v_3
v_2	F	∞	0
v_3	1	0	0
v_4	F	∞	0
v_5	F	∞	0
v_6	F	1	v_3
v_7	F	∞	0
Q	v_1, v_6		



	v_1 Dequeued		
v	*Known*	d_v	p_v
v_1	T	1	v_3
v_2	F	2	v_1
v_3	T	0	0
v_4	F	2	v_1
v_5	F	∞	0
v_6	F	1	v_3
v_7	F	∞	0
Q	v_6, v_2, v_4		

	v_6 Dequeued		
v	*Known*	d_v	p_v
v_1	T	1	v_3
v_2	F	2	v_1
v_3	T	0	0
v_4	F	2	v_1
v_5	F	∞	0
v_6	T	1	v_3
v_7	F	∞	0
Q	v_2, v_4		

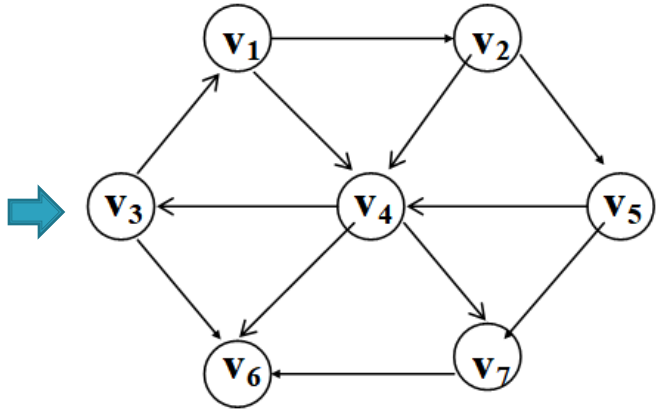
Shortest path: unweighted graph

	v_2 Dequeued		
v	*Known*	d_v	p_v
v_1	T	1	v_3
v_2	T	2	v_1
v_3	T	0	0
v_4	F	2	v_1
v_5	F	3	v_2
v_6	T	1	v_3
v_7	F	∞	0
Q	v_4, v_5		

	v_4 Dequeued		
v	*Known*	d_v	p_v
v_1	T	1	v_3
v_2	T	2	v_1
v_3	T	0	0
v_4	T	2	v_1
v_5	F	3	v_2
v_6	T	1	v_3
v_7	F	3	v_4
Q	v_5, v_7		

	v_5 Dequeued		
v	*Known*	d_v	p_v
v_1	T	1	v_3
v_2	T	2	v_1
v_3	T	0	0
v_4	T	2	v_1
v_5	T	3	v_2
v_6	T	1	v_3
v_7	F	3	v_4
Q	v_7		

	v_7 Dequeued		
v	*Known*	d_v	p_v
v_1	T	1	v_3
v_2	T	2	v_1
v_3	T	0	0
v_4	T	2	v_1
v_5	T	3	v_2
v_6	T	1	v_3
v_7	T	3	v_4
Q	empty		

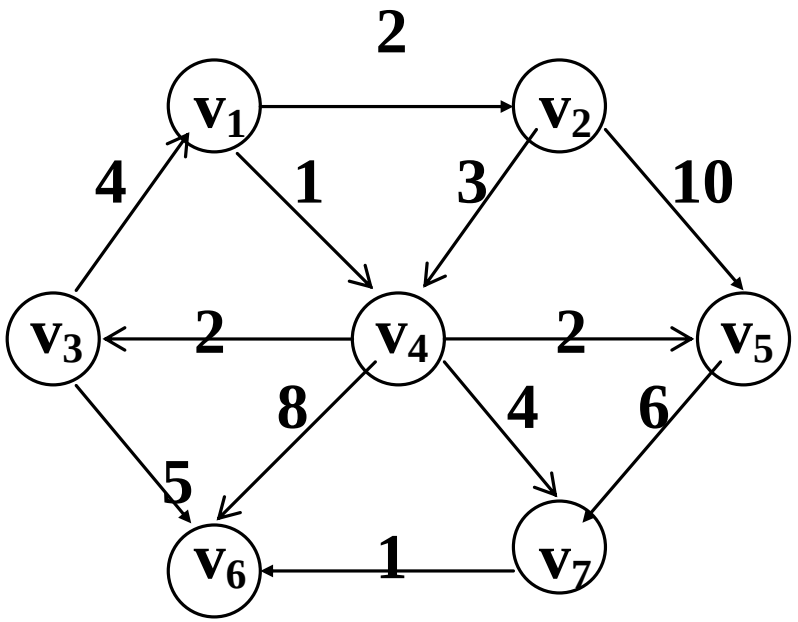


Shortest path: weighted graph

- Dijkstra's algorithm for weighted graphs
  - A greedy algorithm, solving a problem by stages by doing what appears to be the best thing at each stage
  - Select a vertex v, which has the smallest d_v among all the unknown vertices, and declare that the shortest path from s to v is known
  - For each adjacent vertex, w, update $d_w = d_v + c_{v,w}$ if this new value for d_w is an improvement

Example of Dijkstra's algorithm

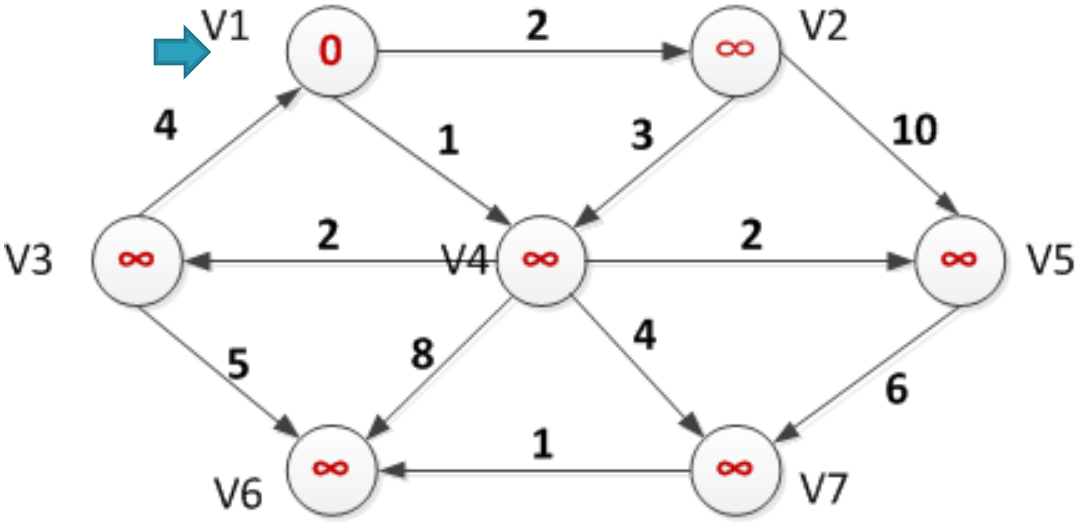
Given a graph, find the shortest path starting from v_1:



Intermediate results of Dijkstra's algo.

v	*Known*	d_v	p_v
v_1	0	0	0
v_2	0	∞	0
v_3	0	∞	0
v_4	0	∞	0
v_5	0	∞	0
v_6	0	∞	0
v_7	0	∞	0

Initial configuration

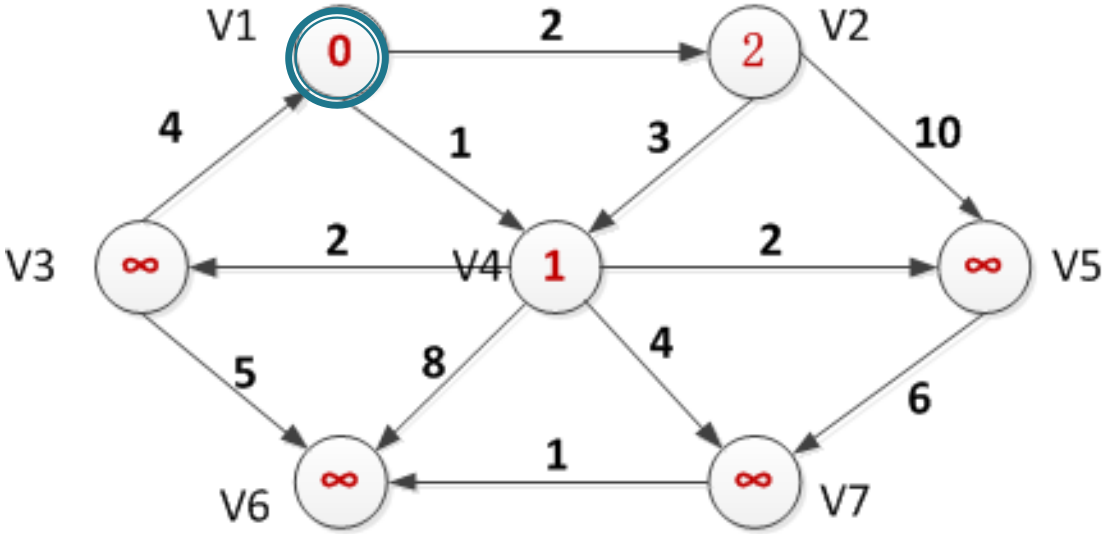


where v_1 is the start vertex

Intermediate results of Dijkstra's algo.

After v_1 is declared known

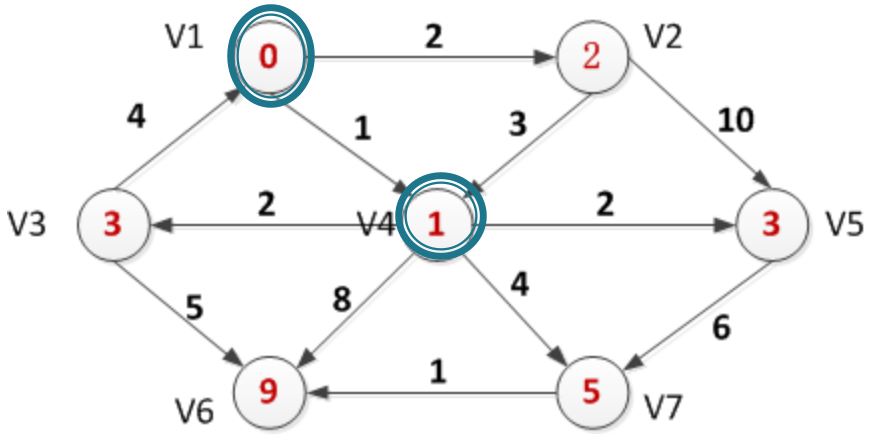
v	*Known*	d_v	p_v
v_1	1	0	0
v_2	0	2	v_1
v_3	0	∞	0
v_4	0	1	v_1
v_5	0	∞	0
v_6	0	∞	0
v_7	0	∞	0



Intermediate results of Dijkstra's algo.

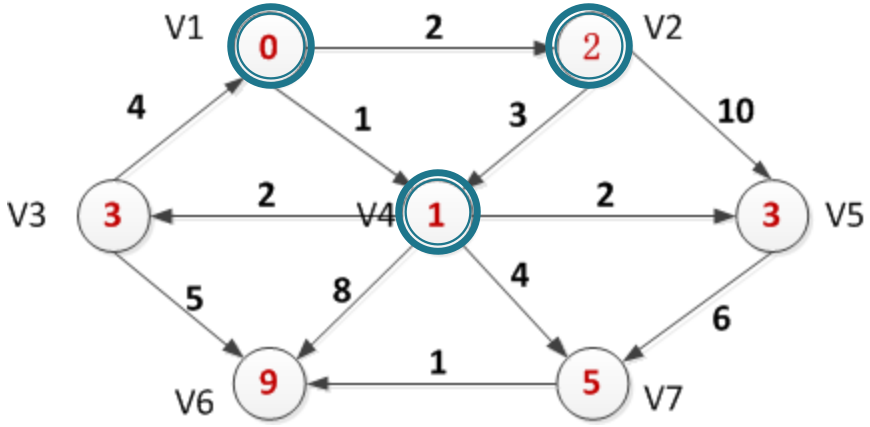
After v_4 is declared known

v	*Known*	d_v	p_v
v_1	1	0	0
v_2	0	2	v_1
v_3	0	3	v_4
v_4	1	1	v_1
v_5	0	3	v_4
v_6	0	9	v_4
v_7	0	5	v_4



After v_2 is declared known

v	*Known*	d_v	p_v
v_1	1	0	0
v_2	1	2	v_1
v_3	0	3	v_4
v_4	1	1	v_1
v_5	0	3	v_4
v_6	0	9	v_4
v_7	0	5	



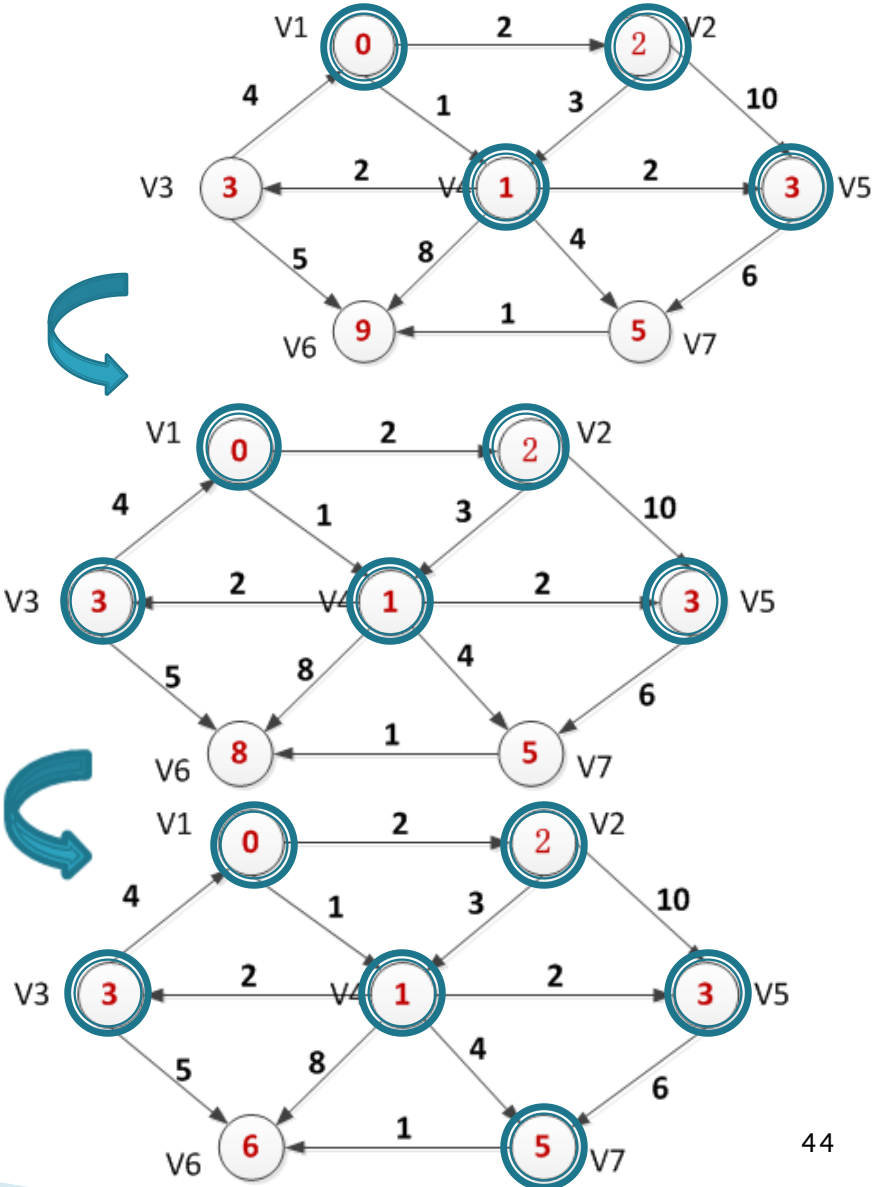
Intermediate results of Dijkstra's algo.

After v_5 and then v_3 are declared known

v	*Known*	d_v	p_v
v_1	1	0	0
v_2	1	2	v_1
v_3	1	3	v_4
v_4	1	1	v_1
v_5	1	3	v_4
v_6	0	8	v_3
v_7	0	5	v_4

After v_7 is declared known

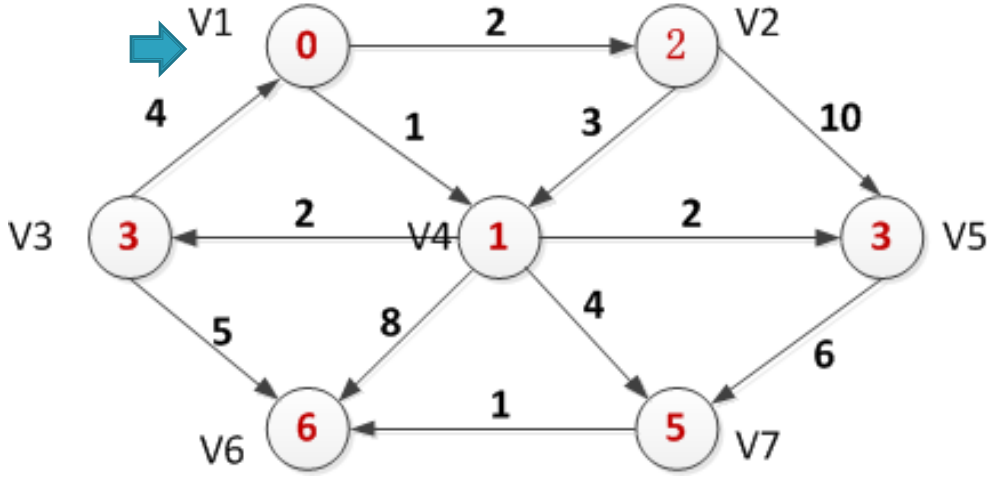
v	*Known*	d_v	p_v
v_1	1	0	0
v_2	1	2	v_1
v_3	1	3	v_4
v_4	1	1	v_1
v_5	1	3	v_4
v_6	0	6	v_7
v_7	1	5	v_4



Intermediate results of Dijkstra's algo.

After v_6 is declared known

v	*Known*	d_v	p_v
v_1	1	0	0
v_2	1	2	v_1
v_3	1	3	v_4
v_4	1	1	v_1
v_5	1	3	v_4
v_6	1	6	v_7
v_7	1	5	v_4



Initialization

Alg.: INITIALIZE-SINGLE-SOURCE(V, s)

1. **for** each $v \in V$
2. **do** $d[v] \leftarrow \infty$
3. $p[v] \leftarrow$ NIL
4. $d[s] \leftarrow 0$

- All the shortest-paths algorithms start with INITIALIZE-SINGLE-SOURCE

Relaxation step

- **Relaxing** an edge (u, v) = testing whether we can improve the shortest path to v found so far by going through u

  If d[v] > d[u] + w(u, v)

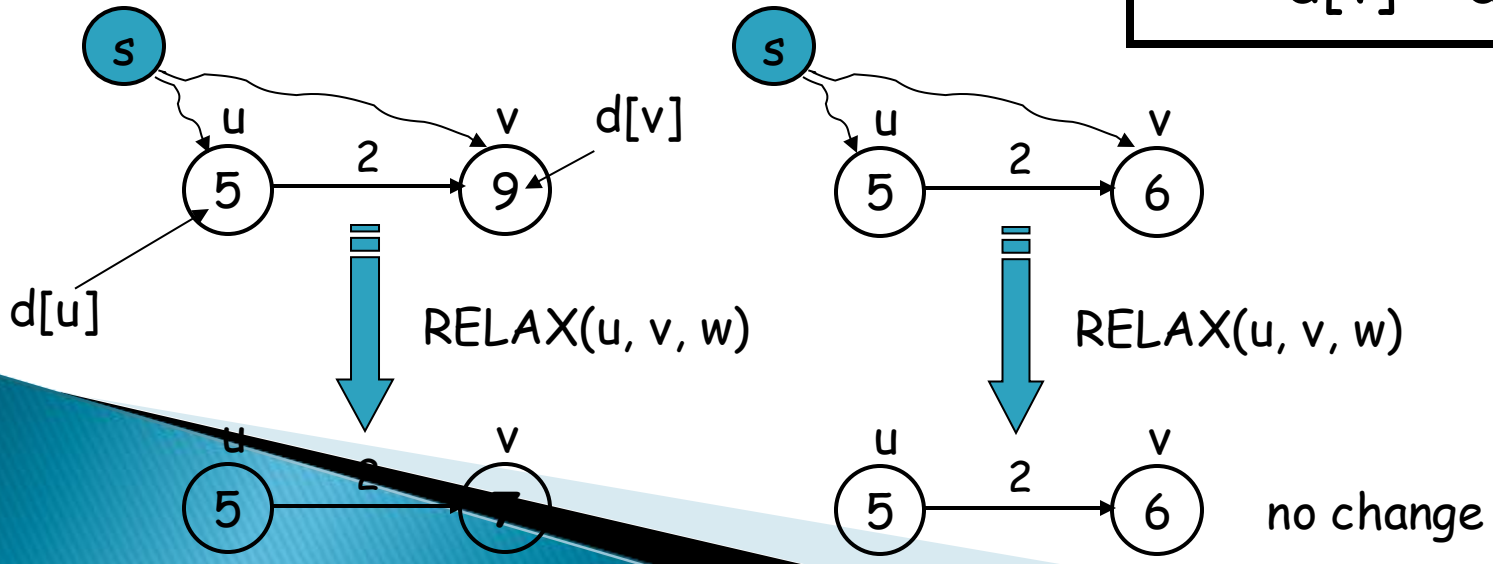
  we can improve the shortest path to v

  ⇒ d[v]=d[u]+w(u,v)

  ⇒ p[v] ← u

After relaxation:

d[v] = d[u] + w(u, v)



Dijkstra(G, w, s)

1. INITIALIZE-SINGLE-SOURCE(V, *s*) ← $\Theta(V)$
2. S ← ∅
3. Q ← V[G] ← $O(V)$ build min-heap
4. **while** Q ≠ ∅ ← Executed $O(V)$ times
5. **do** u ← EXTRACT-MIN(Q) ← $O(\lg V)$ } $O(V \lg V)$
6. S ← S ∪ {u}
7. **for** each vertex v ∈ *A*dj[u] ← $O(E)$ times (total)
8. **do** RELAX(u, v, w)
9. Update Q(DECREASE_KEY) ← $O(\lg V)$ } $O(E \lg V)$

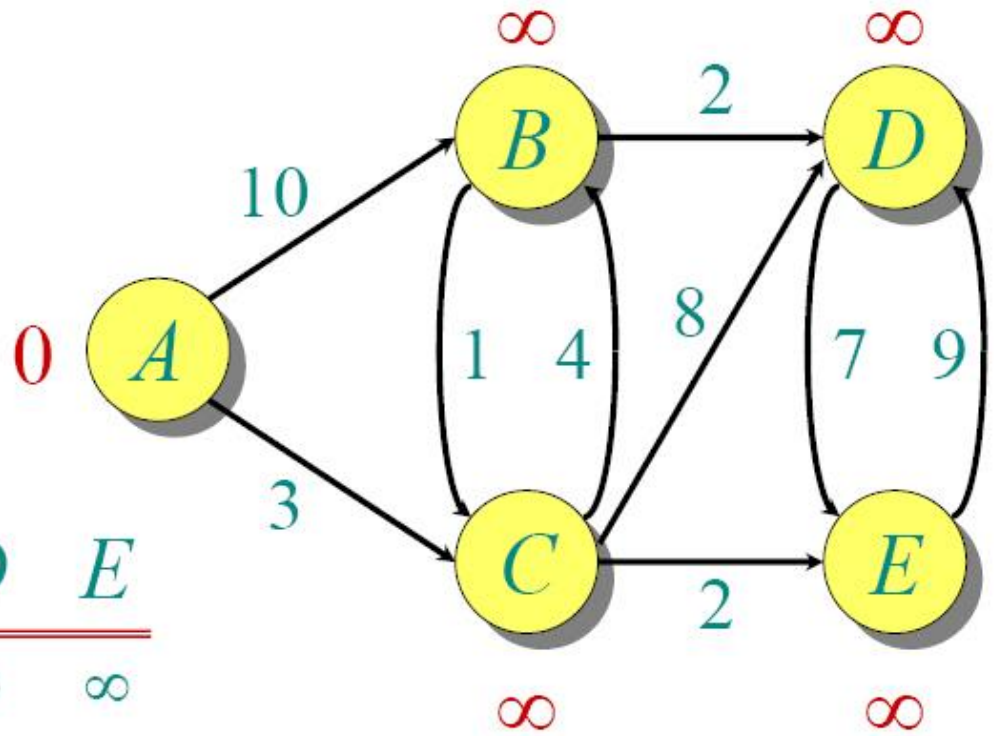
Running time: $O(V \lg V + E \lg V) = O(E \lg V)$

Dijkstra's algorithm – Pseudocode

```
dist[s] ←o                          (distance to source vertex is zero)
for  all v ∈ V–{s}
       do  dist[v] ←∞                (set all other distances to infinity)
S←∅                                 (S, the set of visited vertices is initially empty)
Q←V                                 (Q, the queue initially contains all vertices)
while Q ≠∅                          (while the queue is not empty)
do   u ← mindistance(Q,dist)        (select the element of Q with the min. distance)
     S←SU{u}                        (add u to list of visited vertices)
      for all v ∈ neighbors[u]
            do  if   dist[v] > dist[u] + w(u, v)          (if new shortest path found)
                  then     d[v] ←d[u] + w(u, v)     (set new value of shortest path)
                       (if desired, add traceback code)
return dist
```

Dijkstra Animated Example

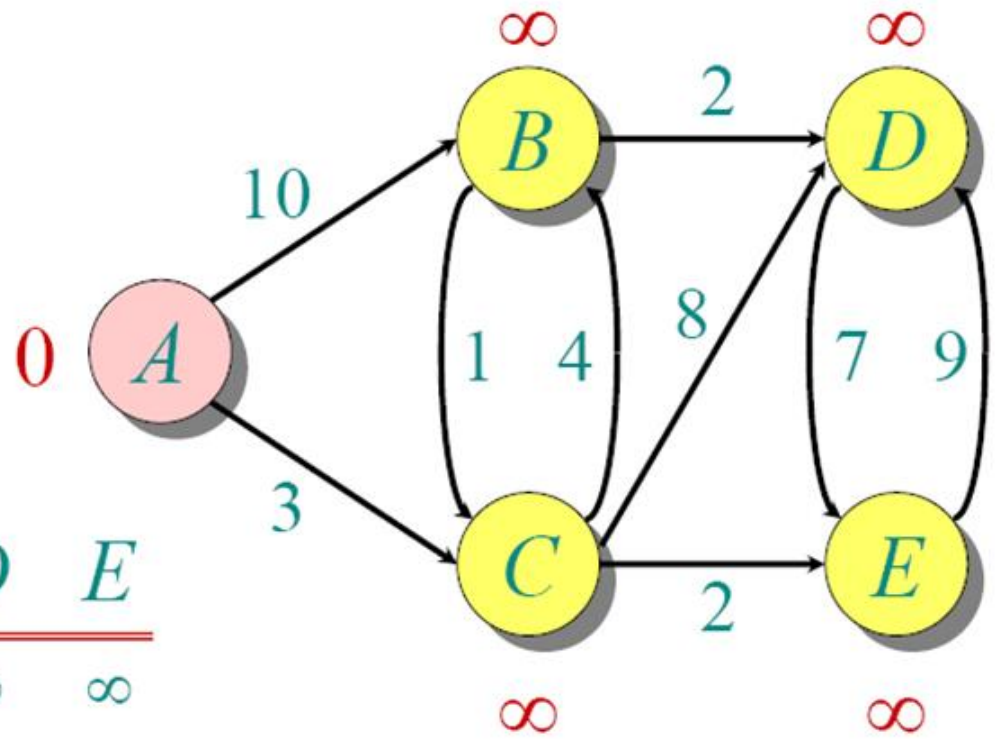
Initialize:



Q:	A	B	C	D	E
	0	∞	∞	∞	∞

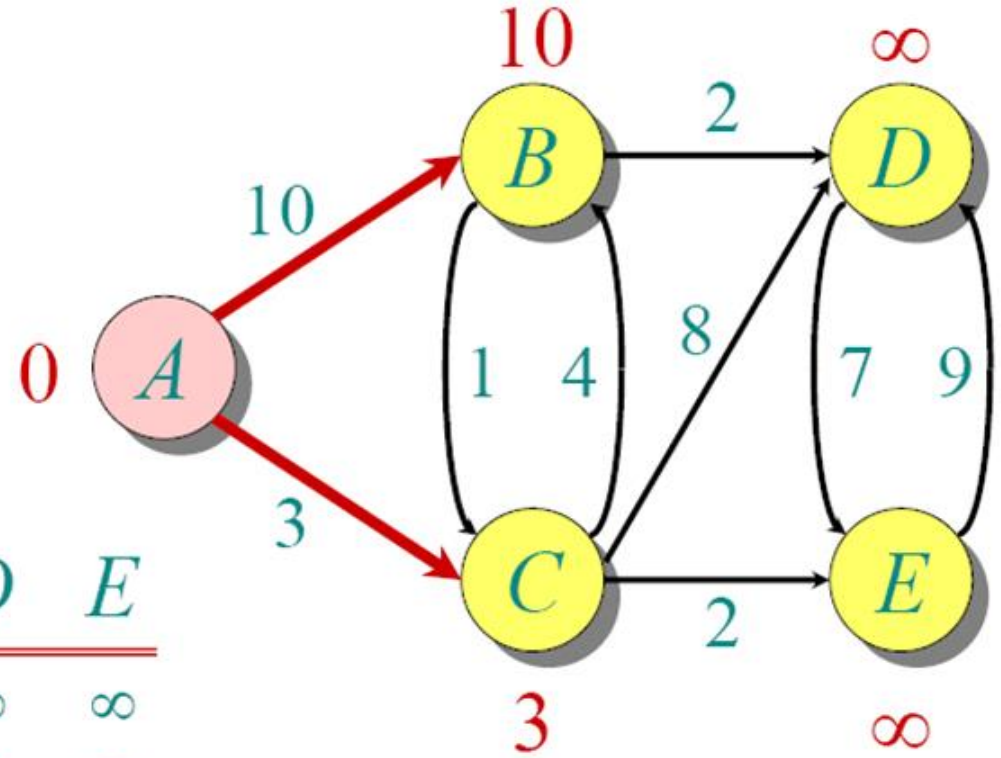
S: {}

Dijkstra Animated Example



Q:	A	B	C	D	E
	0	∞	∞	∞	∞

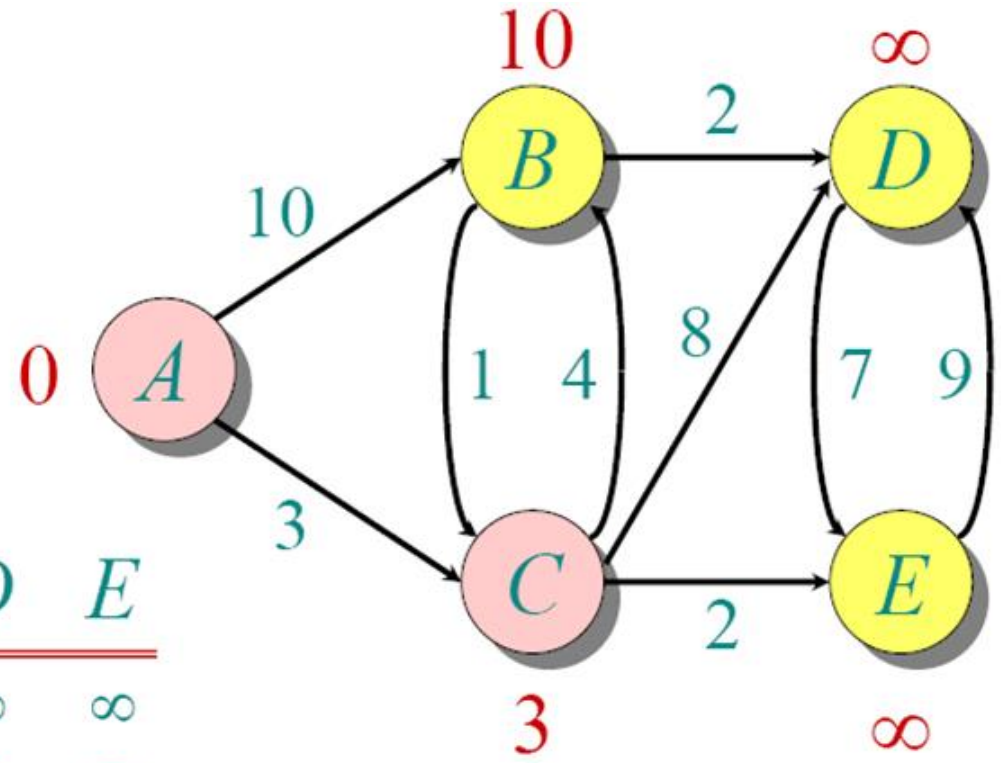
Dijkstra Animated Example



Q:	A	B	C	D	E
	0	∞	∞	∞	∞
		10	3	∞	∞

S: { A }

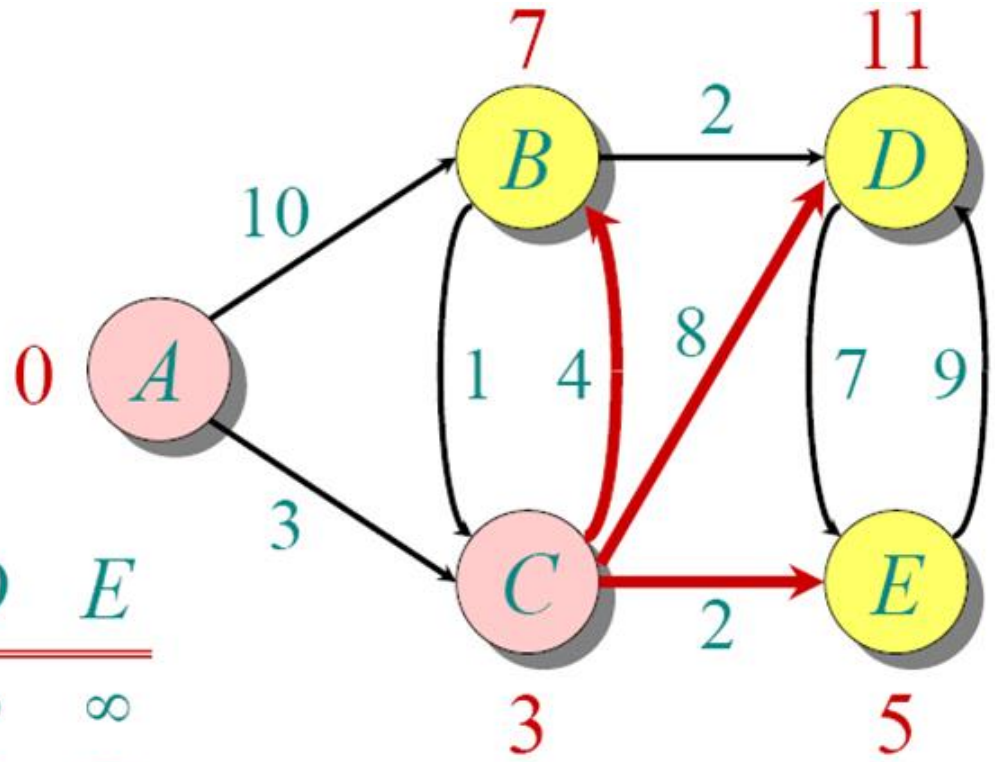
Dijkstra Animated Example



Q:	A	B	C	D	E
	0	∞	∞	∞	∞
		10	3	∞	∞

$S: \{ A, C \}$

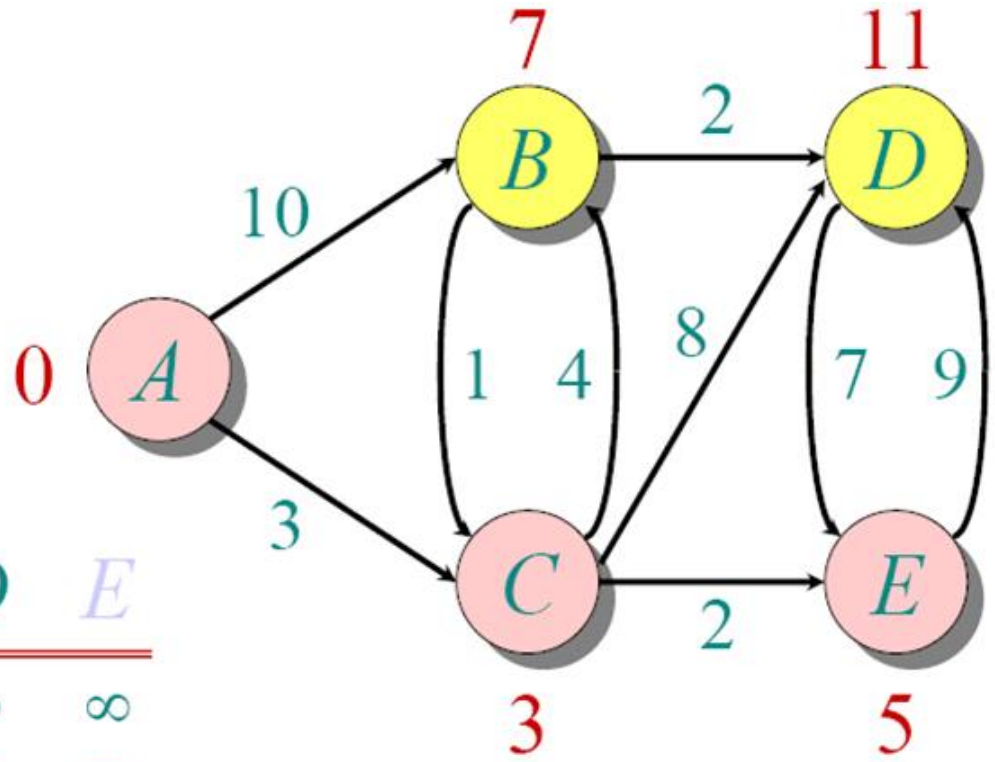
Dijkstra Animated Example



Q:	A	B	C	D	E
	0	∞	∞	∞	∞
		10	3	∞	∞
		7		11	5

$S: \{ A, C \}$

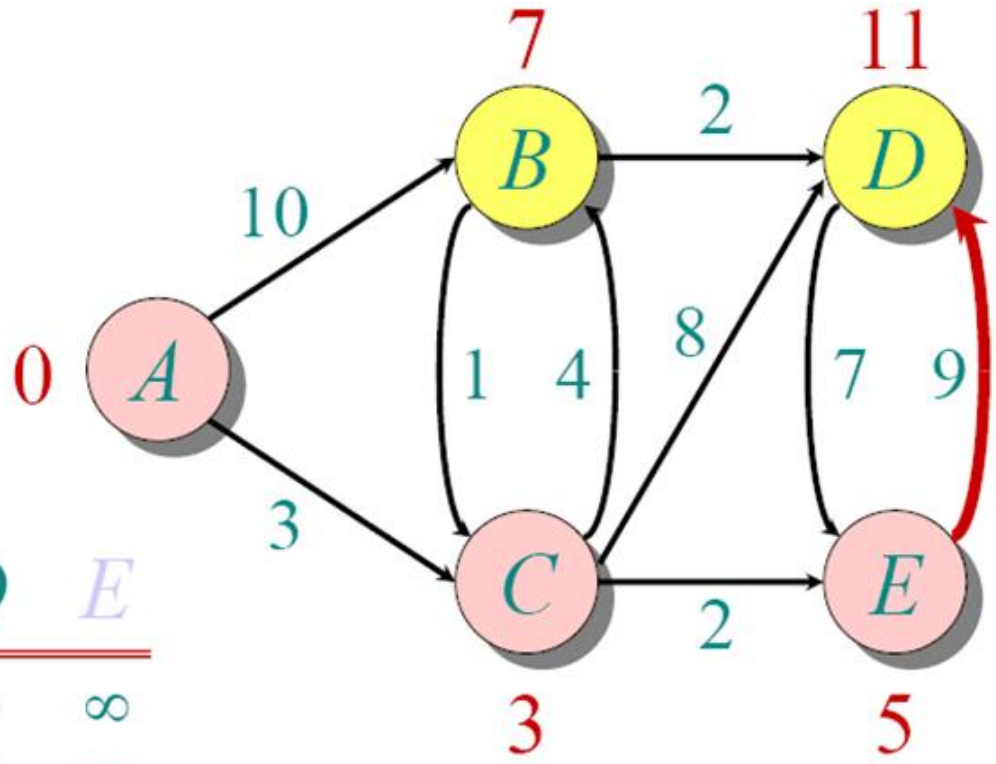
Dijkstra Animated Example



Q:	A	B	C	D	E
	0	∞	∞	∞	∞
		10	3	∞	∞
		7		11	5

$S: \{ A, C, E \}$

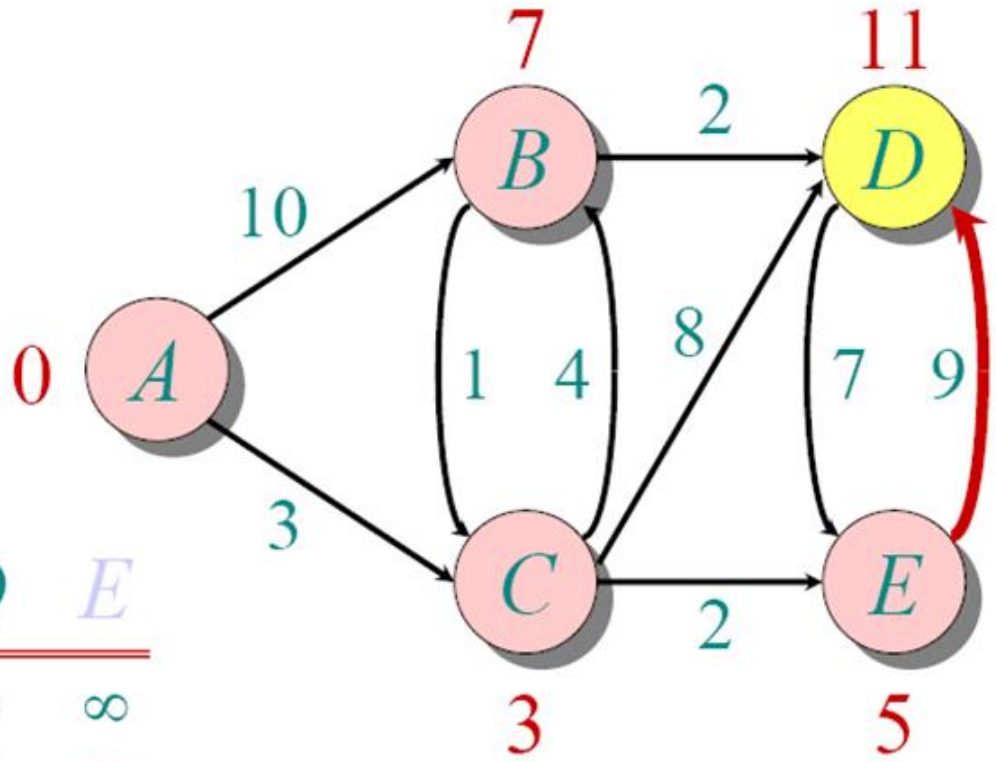
Dijkstra Animated Example



Q:	A	B	C	D	E
	0	∞	∞	∞	∞
		10	3	∞	∞
		7		11	5
		7		11	

S: { A, C, E }

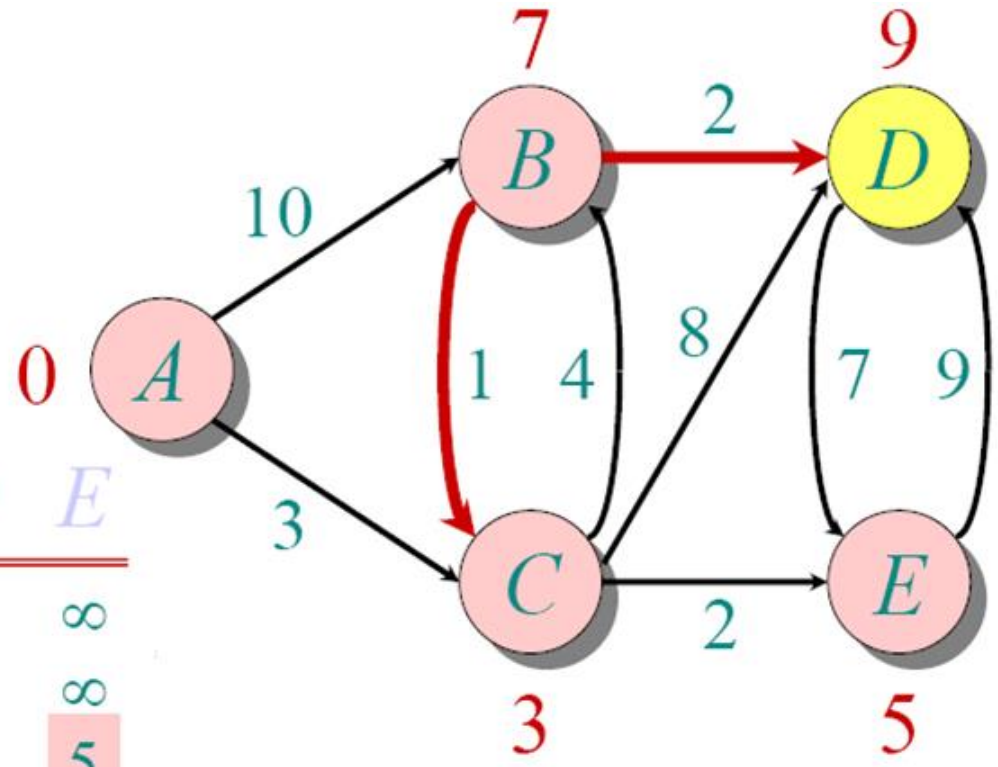
Dijkstra Animated Example



Q:	A	B	C	D	E
	0	∞	∞	∞	∞
		10	3	∞	∞
		7		11	5
		7		11	

S: { A, C, E, B }

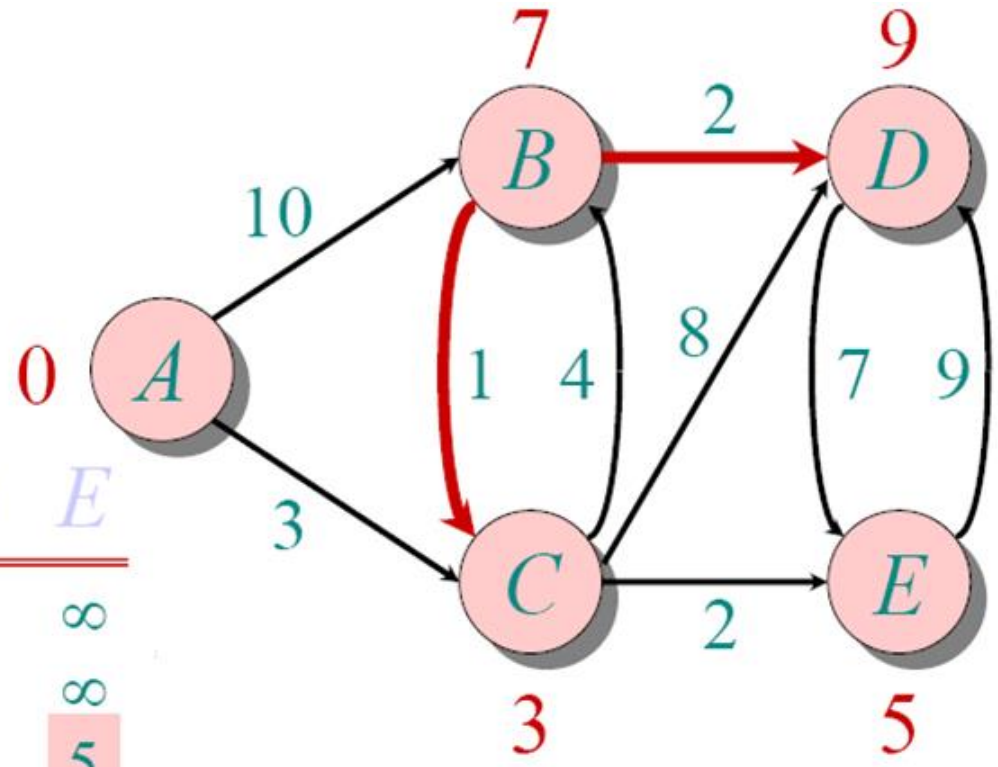
Dijkstra Animated Example



Q:	A	B	C	D	E
	0	∞	∞	∞	∞
		10	3	∞	∞
		7		11	5
		7		11	
				9	

S: { A, C, E, B }

Dijkstra Animated Example



Q:	A	B	C	D	E
	0	∞	∞	∞	∞
		10	3	∞	∞
		7		11	5
		7		11	
				9	

S: { A, C, E, B, D }

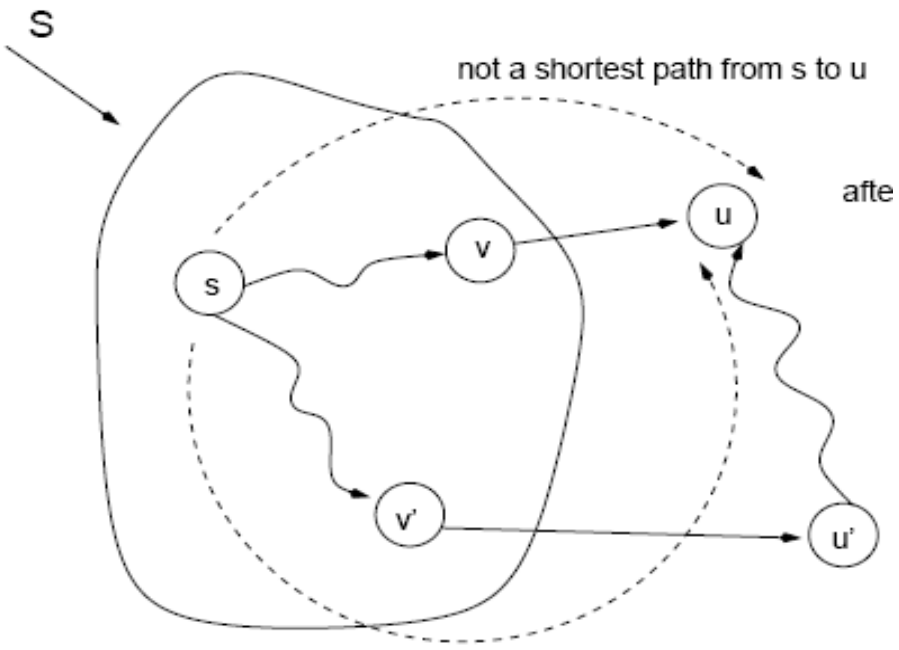
Correctness of Dijkstra's algorithm

- Theorem: For each vertex $u \in V$, we must have $d[u] = \delta(s, u)$ at the time when u is added to S

Proof:

- Assume that u is the first vertex for which $d[u] \neq \delta(s, u)$ when added to S
  - For each vertex v in S, $d[v] = \delta(s, v)$
  - Vertex u has the highest priority in Q: <u, ….> (i.e., d[u] <…)

Correctness of Dijkstra's algorithm



0: If we have a path P' with smaller distance from s to u than d[u], then $\delta(s, u) < d[u]$

1: Let v' be the last vertex in P' such that it is in S, and let u' be the next vertex of v'

2: According to the algorithm, u' must be in the priority queue Q

3: We know $d[u'] < \delta(s, u)$ and further get $d[u'] < \delta(s, u) < d[u]$

4. However, from the assumption, we have $d[u] < d[u']$, so contradiction!

Exercise 1

- Given a directed graph G=(V,E) where each edge (u, v) has an associated value r(u,v), which is a real number in the range $0 \leq r(u,v) \leq 1$ that represents the reliability of a communication channel from vertex u to vertex v
  - We interpret r(u,v) as the probability that the channel from u to v will not fail, and we assume that these probabilities are independent
  - Give an efficient algorithm to find the most reliable path between two given vertices

Exercise 1

- Solution 1: modify Dijkstra's algorithm
  - r(u,v) = Pr(channel from u to v will not fail)
  - Assuming that the probabilities are independent, the reliability of a path $p = \langle v_1, v_2, \ldots, v_k \rangle$ is:

    $r(v_1,v_2)\ r(v_2,v_3)\ \ldots\ r(v_{k-1},v_k)$
  - Find the channel with the highest reliability, i.e.,

$$\max_p \prod_{(u,v)\in p} r(u,v)$$

Exercise 1 (cont.)

- But Dijkstra's algorithm computes

$$\min_p \sum_{(u,v)\in p} w(u,v)$$

- Perform relaxation as follows:

if d[v] $<$ d[u] w(u,v) then
d[v] $=$ d[u] w(u,v)

- Use "EXTRACT_MAX" instead of "EXTRACT_MIN"

Exercise 1 (cont.)

- Solution 2: use Dijkstra's algorithm without any modifications!
  - Goal

$$\max_p \prod_{(u,v)\in p} r(u,v)$$

  - Take the lg

$$\lg(\max_p \prod_{(u,v)\in p} r(u,v)) = \max_p \sum_{(u,v)\in p} \lg(r(u,v))$$

Exercise 1 (cont.)

- Turn this into a minimization problem by taking the negative:

$$-\min_p \sum_{(u,v)\in p} \lg(r(u,v)) = \min_p \sum_{(u,v)\in p} -\lg(r(u,v))$$

- Run Dijkstra's algorithm using

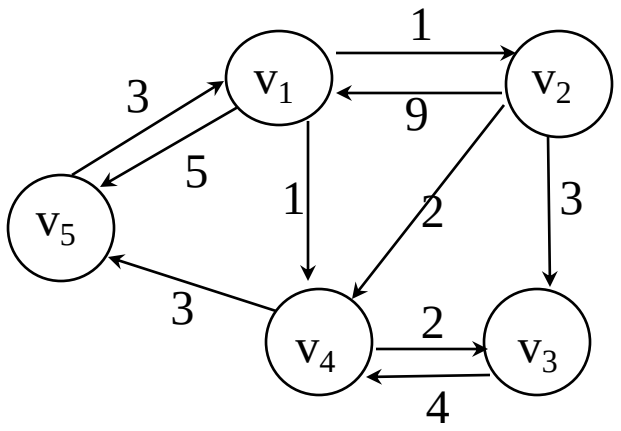
$$w(u,v) = -\lg(r(u,v))$$

Outline

- We focus on weighted graphs
- Graphs with non-negative weights
  - Single-Source Shortest Path: Dijkstra's algorithm
- All-Pair Shortest Path: Floyd's algorithm
- Graphs with negative weights
  - Bellman-Ford algorithm

All pairs shortest path

- A representation: a weight matrix where
  W(i, j) = 0 if i = j
  W(i, j) = ∞ if there is no edge between i and j
  W(i, j) = "weight of edge"



	1	2	3	4	5
1	0	1	∞	1	5
2	9	0	3	2	∞
3	∞	∞	0	4	∞
4	∞	∞	2	0	3
5	3	∞	∞	∞	0

- Problem: find the shortest distance/path between every pair of vertices of a graph

A straightforward method

- A naïve method is to run a single-source shortest path algorithm for each vertex
  - Run Dijkstra's algorithm |V| times
  - Dijkstra's algorithm's time complexity: O(|E| x lg|V|)
  - Total time cost: O(|V| x |E| x lg|V|)

- Floyd's algorithm
  - Total time cost: $O(|V|^3)$
  - For dense graphs, Floyd's algorithm is faster
  - It is easier to implement

Floyd's algorithm

- We have shown principle of optimality applies to shortest path problems
- How can we define the shortest distance $d_{i,j}$ in terms of "smaller" problems?
- Main idea of Floyd's algorithm
  - One way is to restrict the paths to only include vertices from a restricted subset
  - Initially, the subset is empty
  - Then, it is incrementally increased until it includes all the vertices

The subproblems

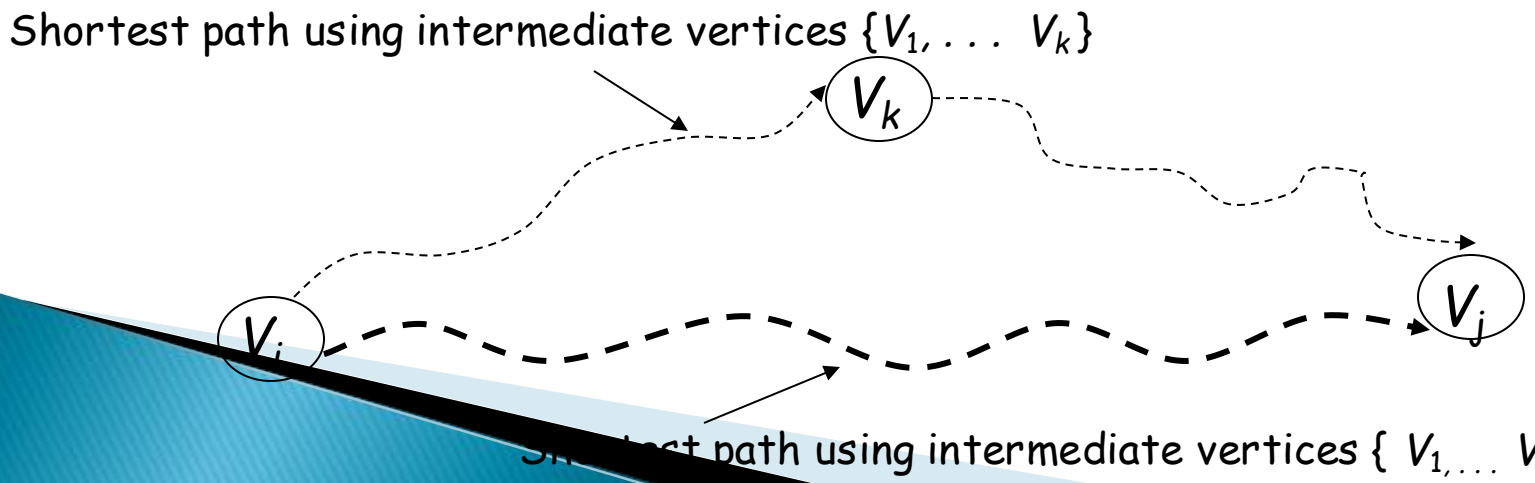
- Let $D^{(k)}[i, j]$ = weight of a shortest path from v_i to v_j using only vertices from $\{v_1, v_2, \ldots, v_k\}$ as intermediate vertices in the path
  - $D^{(0)} = W$
  - $D^{(n)} = D$ which is the goal matrix

- How do we compute $D^{(k)}$ from $D^{(k-1)}$?

The recursive definition:

Case 1: A shortest path from v_i to v_j restricted to using only vertices from $\{v_1, v_2, \ldots, v_k\}$ as intermediate vertices does not use v_k Then $D^{(k)}[i,j] = D^{(k-1)}[i,j]$

Case 2: A shortest path from v_i to v_j restricted to using only vertices from $\{v_1, v_2, \ldots, v_k\}$ as intermediate vertices does use v_k Then $D^{(k)}[i,j] = D^{(k-1)}[i,k] + D^{(k-1)}[k,j]$



The recursive definition

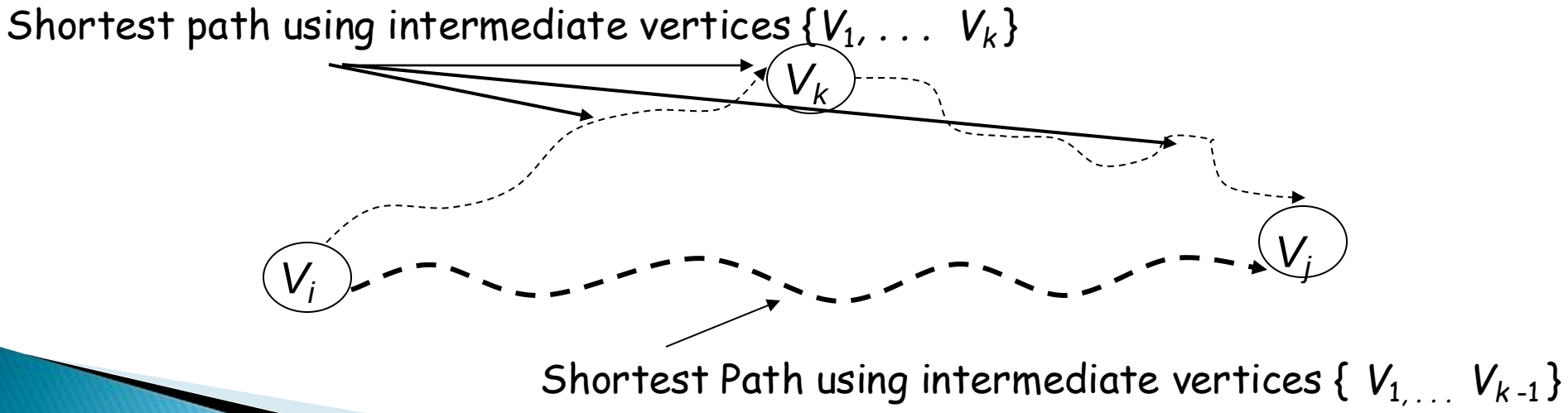
- Since

  $D^{(k)}[i, j] = D^{(k-1)}[i, j]$ or

  $D^{(k)}[i, j] = D^{(k-1)}[i, k] + D^{(k-1)}[k, j]$

  We conclude:

  $D^{(k)}[i, j] = \min\{ D^{(k-1)}[i, j], D^{(k-1)}[i, k] + D^{(k-1)}[k, j] \}$



The pointer array P

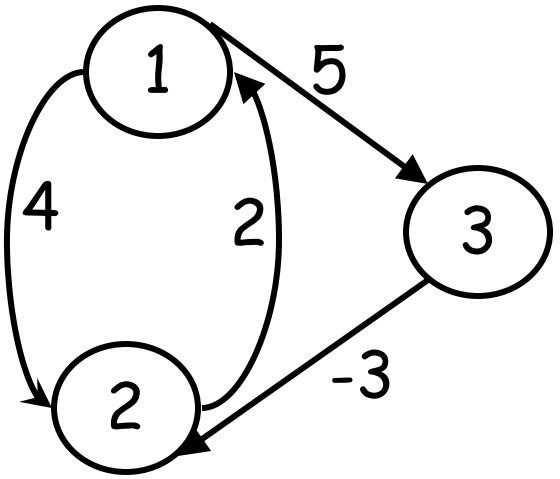
- How to recover the shortest paths?
  - We can use a pointer array P, which initially contains 0
  - Each time that a shorter path from *i* to *j* is found, the *k* that provided the minimum distance is saved
  - To print the intermediate nodes on the shortest path by a recursive procedure, which print the shortest paths from *i* and *k*, and from *k* to *j*

Floyd's algorithm using n+1 *D* matrices

Floyd //Computes shortest distance between all pairs of
    //nodes, and saves P to enable finding shortest paths

1. $D^0 \leftarrow W$ // initialize *D* array to *W* [ ]
2. $P \leftarrow 0$ // initialize *P* array to [0]
3. for $k \leftarrow 1$ to n
4. do for $i \leftarrow 1$ to n
5. do for $j \leftarrow 1$ to n
6. if ($D^{k-1}[\, i, j\,] > D^{k-1}\,[\, i, k\,] + D^{k-1}\,[\, k, j\,]$)
7. then $D^k[\, i, j\,] \leftarrow D^{k-1}\,[\, i, k\,] + D^{k-1}\,[\, k, j\,]$
8. $P[\, i, j\,] \leftarrow k$;
9. else $D^k[\, i, j\,] \leftarrow D^{k-1}\,[\, i, j\,]$

Example



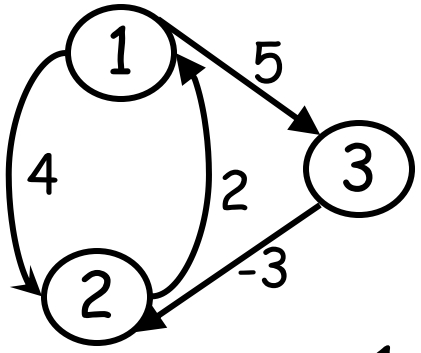
$W = D^0 =$

	1	2	3
1	0	4	5
2	2	0	∞
3	∞	-3	0

$P =$

	1	2	3
1	0	0	0
2	0	0	0
3	0	0	0

Example



D^0 =

	1	2	3
1	0	4	5
2	2	0	∞
3	∞	-3	0

k = 1
Vertex 1 can be intermediate node

D^1 =

	1	2	3
1	0	4	5
2	2	0	7
3	∞	-3	0

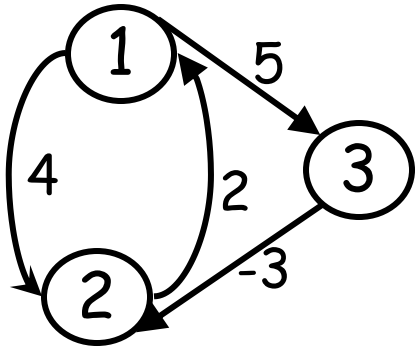
$D^1[2,3] = \min(D^0[2,3], D^0[2,1]+D^0[1,3])$
$= \min (\infty, 7)$
$= 7$

P =

	1	2	3
1	0	0	0
2	0	0	1
3	0	0	0

$D^1[3,2] = \min(D^0[3,2], D^0[3,1]+D^0[1,2])$
$= \min (-3,\infty)$
$= -3$

Example



D^1 =

	1	2	3
1	0	4	5
2	2	0	7
3	∞	-3	0

k = 2
Vertices 1, 2 can be intermediate

D^2 =

	1	2	3
1	0	4	5
2	2	0	7
3	-1	-3	0

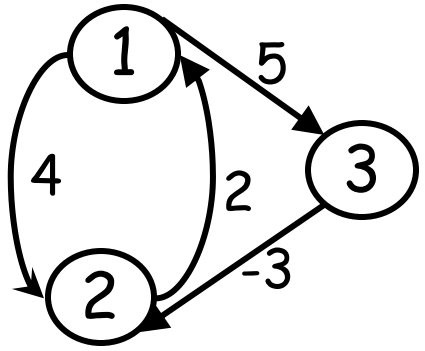
$D^2[1,3] = \min(D^1[1,3], D^1[1,2]+D^1[2,3])$
$= \min (5, 4+7)$
$= 5$

P =

	1	2	3
1	0	0	0
2	0	0	1
3	2	0	0

$D^2[3,1] = \min(D^1[3,1], D^1[3,2]+D^1[2,1])$
$= \min (\infty, -3+2)$
$= -1$

Example



D^2 =

	1	2	3
1	0	4	5
2	2	0	7
3	-1	-3	0

k = 3
Vertices 1, 2, 3 can be intermediate

D^3 =

	1	2	3
1	0	2	5
2	2	0	7
3	-1	-3	0

$D^3[1,2] = \min(D^2[1,2], D^2[1,3]+D^2[3,2])$
$= \min(4, 5+(-3))$
$= 2$

P =

	1	2	3
1	0	3	0
2	0	0	1
3	2	0	0

$D^3[2,1] = \min(D^2[2,1], D^2[2,3]+D^2[3,1])$
$= \min(2, 7+(-1))$
$= 2$

Floyd's algorithm using 2 *D* matrices

Floyd's algorithm

1. $D \leftarrow W$ // initialize D array to $W[\ ]$
2. $P \leftarrow 0$ // initialize P array to [0]
3. for $k \leftarrow 1$ to n

   // Computing D' from D

4. do for $i \leftarrow 1$ to n
5. do for $j \leftarrow 1$ to n
6. if ($D[\ i, j\ ] > D[\ i, k\ ] + D[\ k, j\ ]$)
7. then $D'[\ i, j\ ] \leftarrow D[\ i, k\ ] + D[\ k, j\ ]$
8. $P[\ i, j\ ] \leftarrow k$;
9. else $D'[\ i, j\ ] \leftarrow D[\ i, j\ ]$
10. *Move D' to D*

Can we use only one D matrix?

- $D[i, j]$ depends only on elements in the k–th column and row of the distance matrix
- We will show that the k–th row and the k–th column of the distance matrix are unchanged when D^k is computed
- This means D can be calculated *in–place*

The main diagonal values

- Before we show that k-th row and column of D remain unchanged we show that the main diagonal remains 0

- $D^{(k)}[j, j] = \min\{ D^{(k-1)}[j, j], D^{(k-1)}[j, k] + D^{(k-1)}[k, j] \}$
  $= \min\{ 0, \ D^{(k-1)}[j, k] + D^{(k-1)}[k, j] \}$
  $= 0$

The *k*th column

- k-th column of D^k is equal to the k-th column of D^{k-1}

- Intuitively true – a path from i to k will not become shorter by adding k to the allowed subset of intermediate vertices

- For all i, $D^{(k)}[i,k] =$
  $= \min\{ D^{(k-1)}[i,k],\ D^{(k-1)}[i,k] + D^{(k-1)}[k,k] \}$
  $= \min \{ D^{(k-1)}[i,k], D^{(k-1)}[i,k] + 0 \}$
  $= D^{(k-1)}[i,k]$

The *k*th row

- k-th row of D^k is equal to the kth row of D^{k-1}

For all j, $D^{(k)}[k,j] =$

$= \min\{ D^{(k-1)}[k, j], D^{(k-1)}[k, k] + D^{(k-1)}[k, j]$

}

$= \min\{ D^{(k-1)}[k, j], 0 + D^{(k-1)}[k, j] \}$

$= D^{(k-1)}[k, j]$

Question

- Can we claim that D^k equals to D^{k-1}, D^{k-2} ?
  - No, we can only claim
    - The 1-st row and 1-st column of D^1 equal to the 1-st row and 1-st column of D^0, respectively
    - The 2-nd row and 2-nd column of D^2 equal to the 2-nd row and 2-nd column of D^1, respectively
    - ……

Floyd's algorithm using a single D

```
Floyd
  1. D ← W   // initialize D array to W[ ]
  2. P ← 0   // initialize P array to [0]
  3. for k ← 1 to n
  4.       do for i ← 1 to n
  5.             do for j ← 1 to n
  6.                   if (D[ i, j ] > D[ i, k ] + D[ k, j ] )
  7.                         then  D[ i, j ] ← D[ i, k ] + D[ k, j ]
  8.                               P[ i, j ] ← k;
```

$O(|V|^3)$

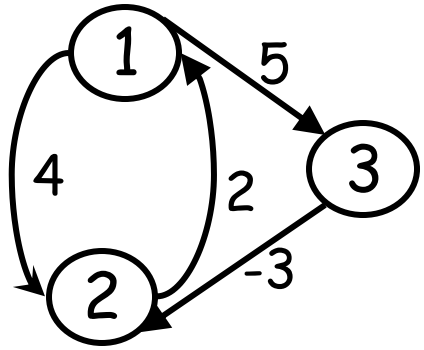
Total time cost: $O(|V|^3)$

Printing intermediate nodes on shortest path from q to r

```
path(index q, r)
  if (P[ q, r ] != 0)
        path(q, P[q, r])
        println( "v"+ P[q, r])
        path(P[q, r], r)
         return;
  //no intermediate nodes
  else return
```

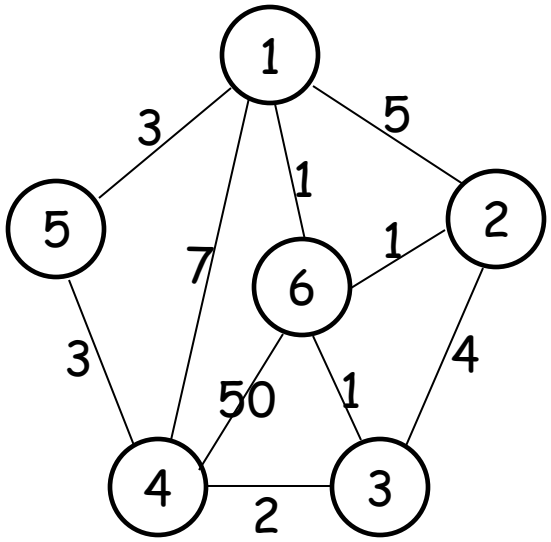
P =

	1	2	3
1	0	3	0
2	0	0	1
3	2	0	0



Before calling path check D[q, r] $< \infty$, and print node q, after the call to path print node r

Example

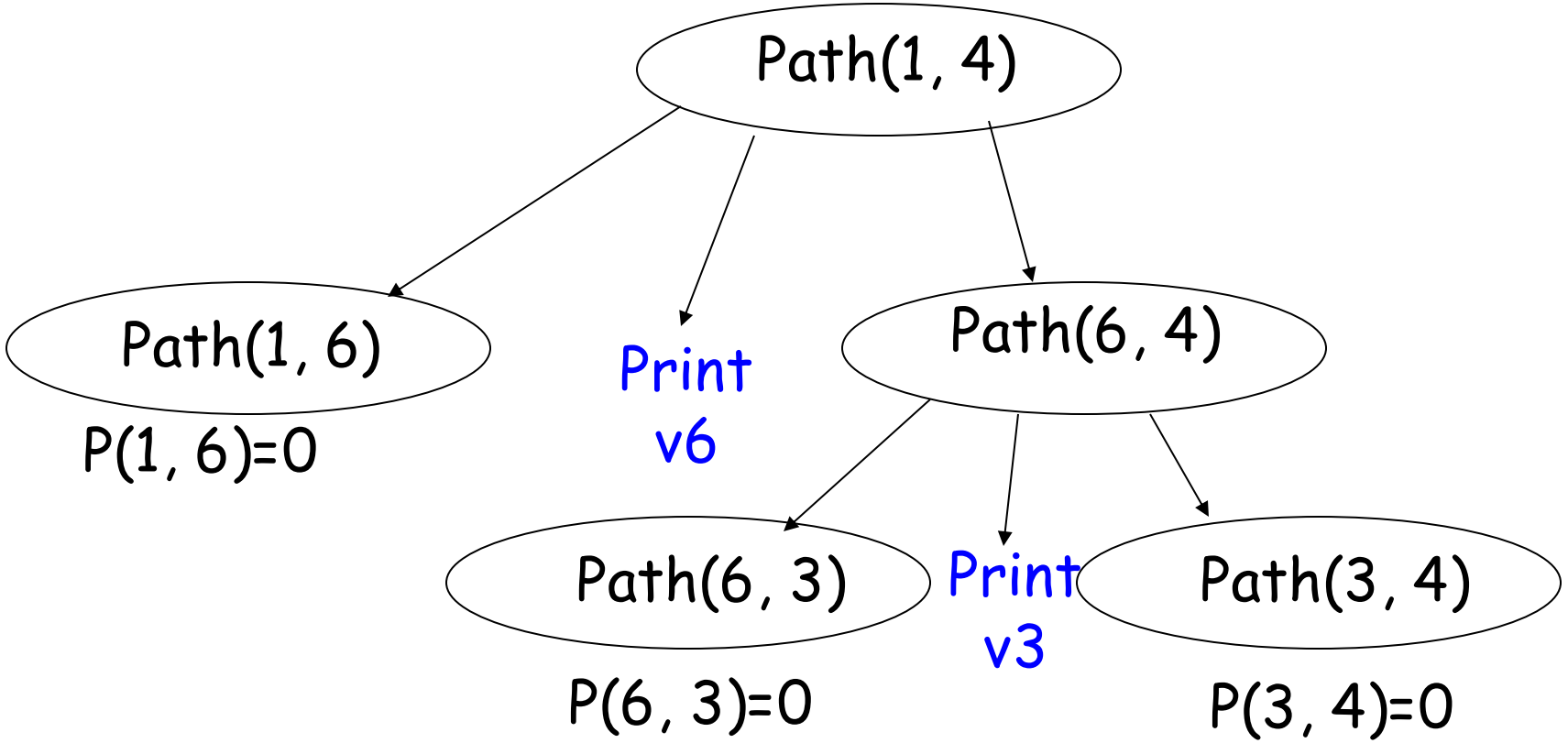


$D^6 =$

	1	2	3	4	5	6
1	0	2(6)	2(6)	4(6)	3	1
2	2(6)	0	2(6)	4(6)	5(6)	1
3	2(6)	2(6)	0	2	5(4)	1
4	4(6)	4(6)	2	0	3	3(3)
5	3	5(6)	5(4)	3	0	4(1)
6	1	1	1	3(3)	4(1)	0

The values in parenthesis are the non-zero P values

The call tree for Path(1, 4)



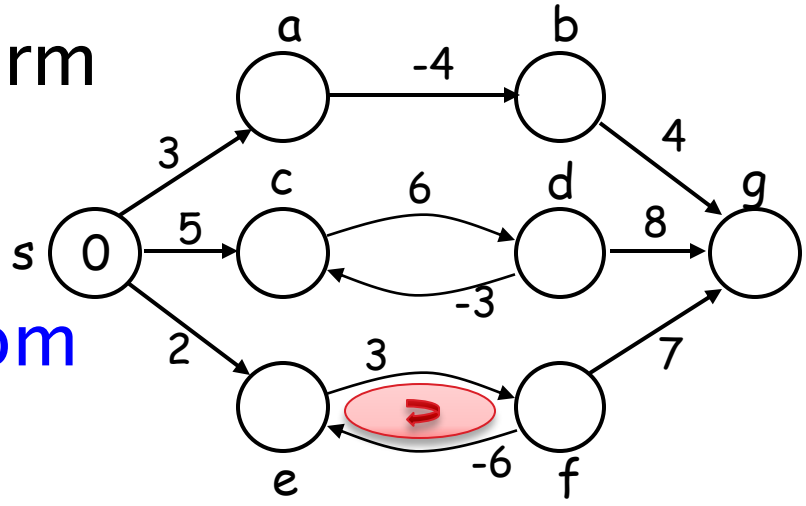
The intermediate nodes on the shortest path from v1 to v4 are v6, v3, so the shortest path is v1, v6, v3, v4.

Outline

- Shortest path problem
  - Graphs with non-negative weights
    - Single-Source Shortest Path: Dijkstra's algorithm
  - All-Pair Shortest Path: Floyd's algorithm
  - Graphs with negative weights
    - Bellman-Ford algorithm

Negative-weight edges

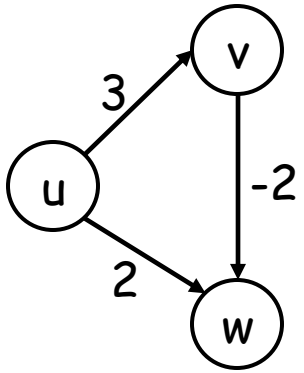
- Negative-weight edges may form negative-weight cycles



- If such cycles are reachable from the source, then δ(s, v) is not properly defined!
  - Keep going around the cycle, and get $w(s, v) = -\infty$ for all v on the cycle

Is Dijkstra's algorithm still applicable for graphs with negative weights?

- Dijkstra's algorithm cannot handle a graph that has negative weights but no negative cycles



- How to handle a graph that has negative weights but no negative cycles?

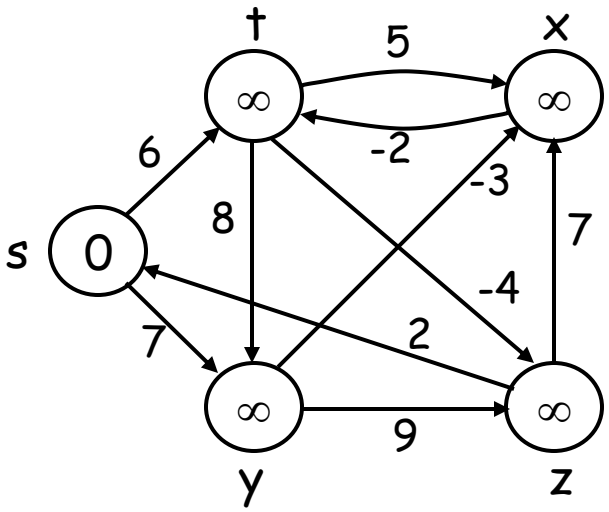
Bellman-Ford algorithm

- Single-source shortest path problem
  - Computes $\delta(s, v)$ and $p[v]$ for all $v \in V$

- Allows negative edge weights – can detect negative cycles
  - Returns TRUE if no negative-weight cycles are reachable from the source s
  - Returns FALSE otherwise $\Rightarrow$ no solution exists

Bellman-Ford algorithm (cont'd)

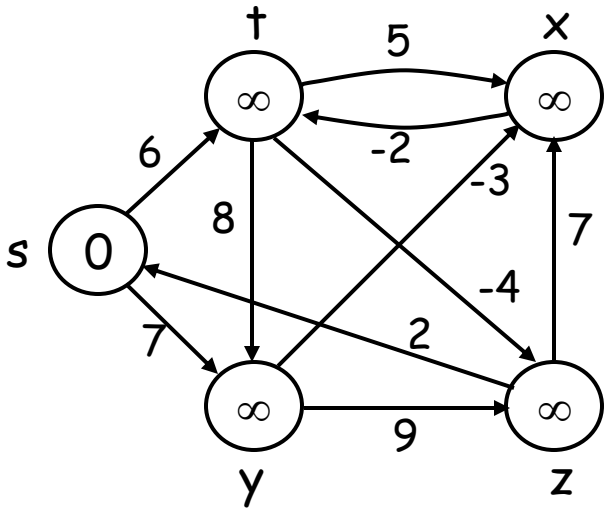
- Idea:
  - Each edge is relaxed |V| – 1 times by making |V| –1 passes over the whole edge set
  - Any path will contain at most |V| – 1 edges



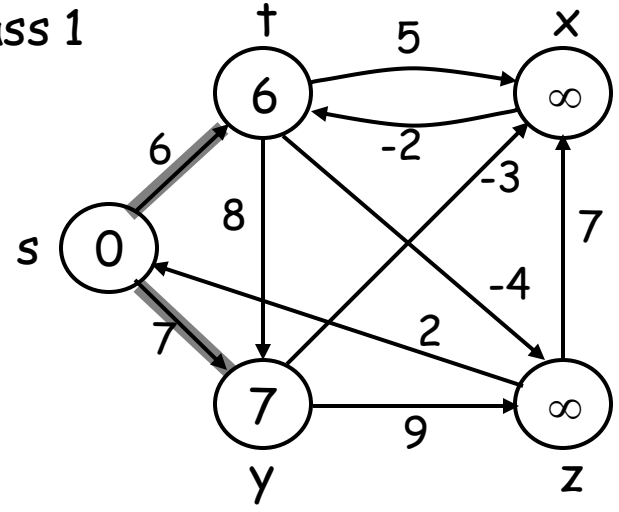
For each edge (u, v), do relaxation:
If d[v] > d[u] + w(u, v)
⇒ d[v] = d[u] + w(u, v)

Edge order: (t, x), (t, y), (t, z), (x, t), (y, x), (y, z), (z, x), (z, s), (s, t), (s, y)

Bellman-Ford(V, E, w, s)



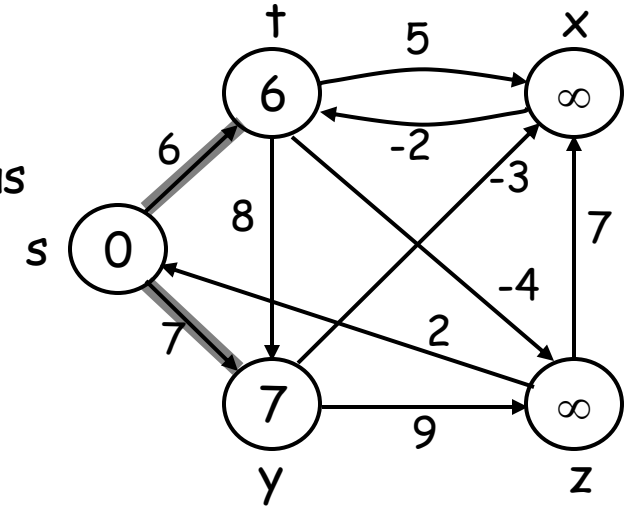
Pass 1



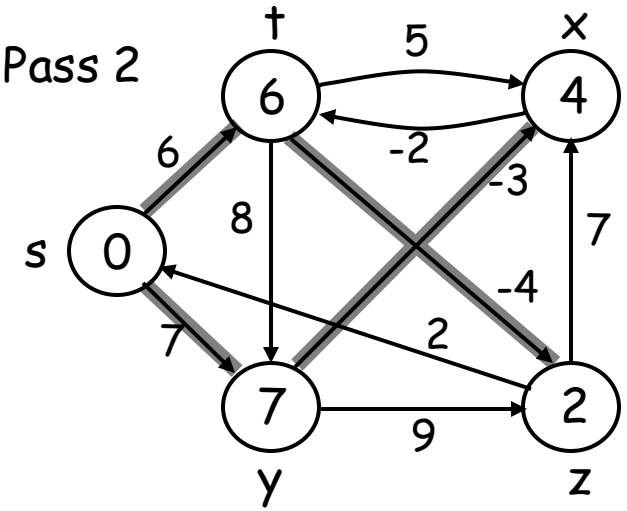
Edge order: (t, x), (t, y), (t, z), (x, t), (y, x), (y, z), (z, x), (z, s), (s, t), (s, y)

Example

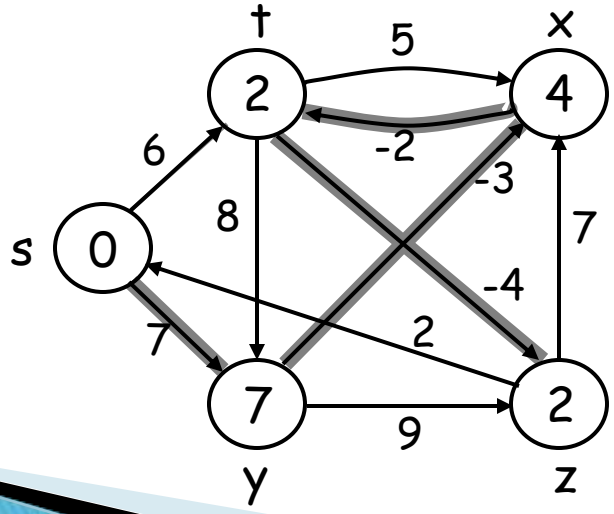
Pass 1 (from previous slide)



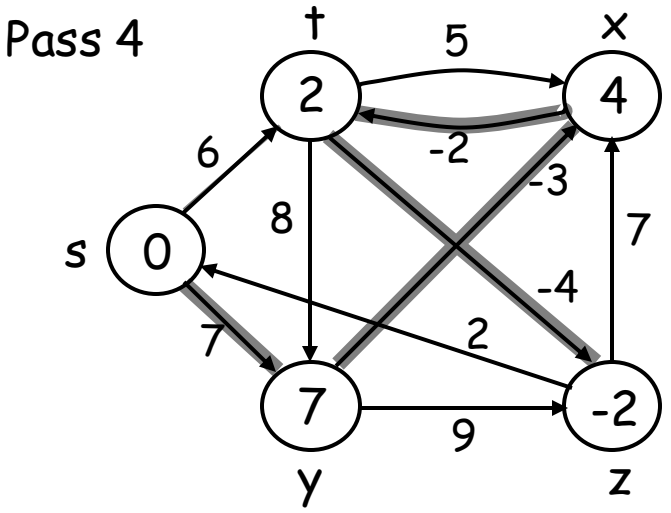
Pass 2



Pass 3



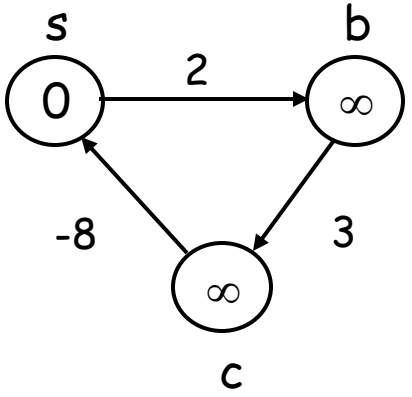
Pass 4

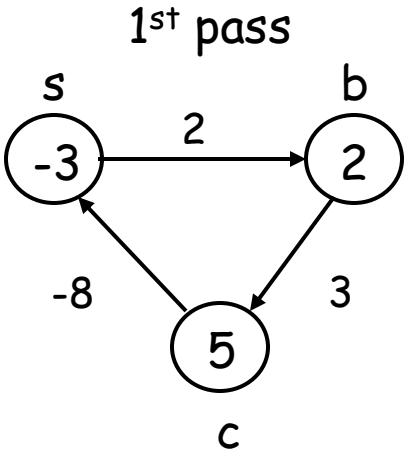


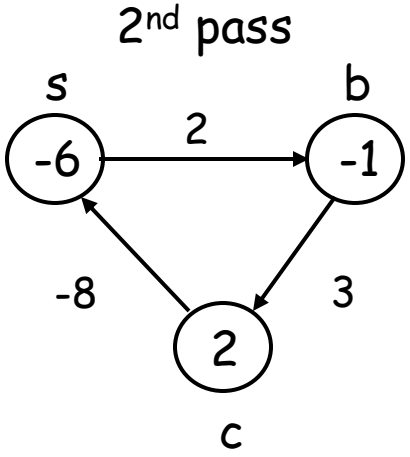
Edge order: (t, x), (t, y), (t, z), (x, t), (y, x), (y, z), (z, x), (z, s), (s, t), (s, y)

Detecting negative cycles (perform extra test after |V|−1 iterations)

- **for** each edge (u, v) ∈ E
- **do if** d[v] > d[u] + w(u, v)
- **then return** FALSE
- **return** TRUE







Look at edge (s, b):

d[b] = -1
d[s] + w(s, b) = -4

⇒ d[b] > d[s] + w(s, b)

(s, b), (b, c), (c, s)

Bellman-Ford(V, E, w, s)

```
1.  INITIALIZE-SINGLE-SOURCE(V, s)          ← Θ(|V|)
2.  for i ← 1 to |V| - 1                    ← O(|V|)
3.     do for each edge (u, v) ∈ E          ← O(|E|)      } O(|V||E|)
4.           do RELAX(u, v, w)
5.  for each edge (u, v) ∈ E                ← O(|E|)
6.     do if d[v] > d[u] + w(u, v)
7.           then return FALSE
8.   return TRUE
```

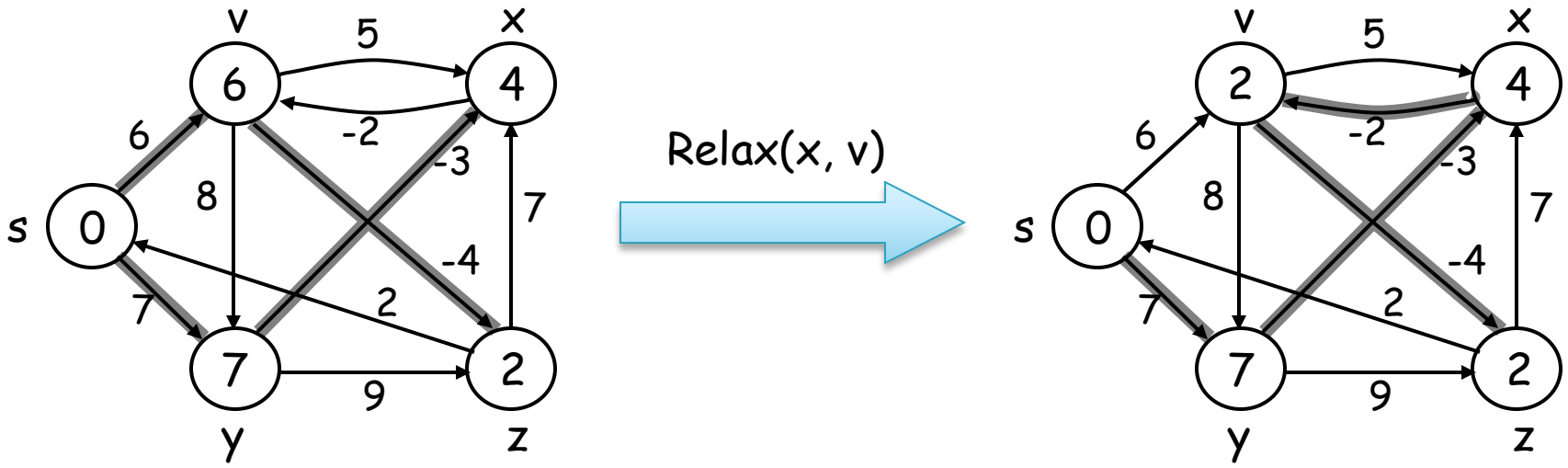
Running time: O(|V||E|)

Key points of Bellman-Ford

- If there is no negative cycle, after |V|-1 iterations, d values will not be updated or can't be lower any more, and d values store the measure of the shortest path
  - Why? How to prove its correctness?

Shortest path properties

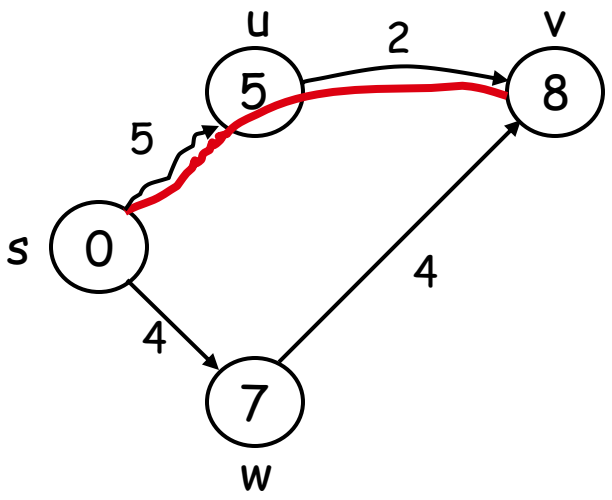
- **Upper-bound property**
  - We always have $d[v] \geq \delta(s, v)$ for all v
  - The estimate never goes up – relaxation only lowers the estimate



Shortest path properties

- **Convergence property**

  If $s \rightsquigarrow u \rightarrow v$ is a shortest path, and if $d[u] = \delta(s, u)$ at any time prior to relaxing edge (u, v), then $d[v] = \delta(s, v)$ at all times after relaxing (u, v)



• If $d[v] > \delta(s, v) \Rightarrow$ after relaxation:

$d[v] = d[u] + w(u, v)$

$d[v] = 5 + 2 = 7$

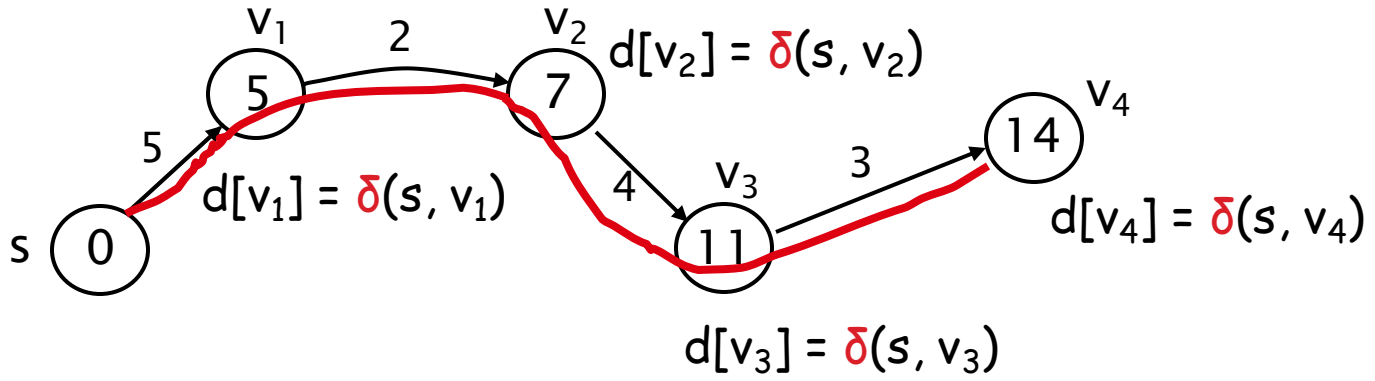
• Otherwise, the value remains unchanged, because it must have been the shortest path value

Shortest path properties

- **Path relaxation property**

  Let $p=\langle v_0, v_1, \ldots, v_k \rangle$ be a shortest path from $s = v_0$ to v_k

  If we relax, in order, $(v_0, v_1), (v_1, v_2), \ldots, (v_{k-1}, v_k)$, even intermixed with other relaxations, then $d[v_k] = \delta(s, v_k)$



Correctness of Bellman-Ford algorithm

- **Theorem:** Show that $d[v] = \delta(s, v)$, for every v, after $|V| - 1$ passes

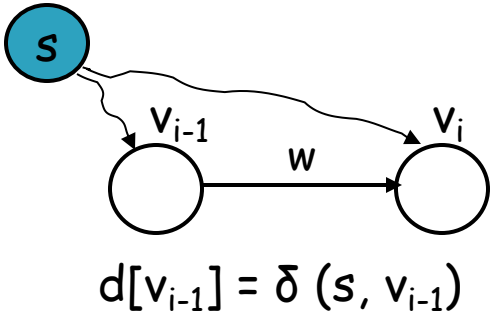
- Case 1: G does not contain negative cycles which are reachable from s

  - Assume that the shortest path from s to v is
    $p = \langle v_0, v_1, \ldots, v_k \rangle$, where $s = v_0$ and $v = v_k$, $k \leq |V|-1$

  - Use mathematical induction on the number of passes i to show that:
    $d[v_i] = \delta(s, v_i)$, $i=0,1,\ldots,k$

Correctness of Bellman-Ford algorithm

Base case: $i=0$, $d[v_0] = \delta(s, v_0) = \delta(s, s) = 0$
Inductive hypothesis: $d[v_{i-1}] = \delta(s, v_{i-1})$
Inductive step: $d[v_i] = \delta(s, v_i)$



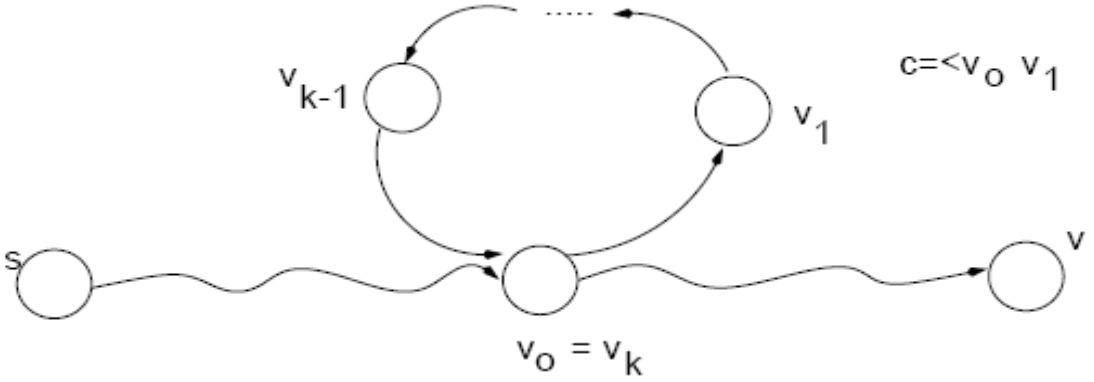
After relaxing (v_{i-1}, v_i) (convergence property) :
$d[v_i] \leq d[v_{i-1}] + w = \delta(s, v_{i-1}) + w = \delta(s, v_i)$

From the upper bound property: $d[v_i] \geq \delta(s, v_i)$

Therefore, $d[v_i] = \delta(s, v_i)$

Correctness of Bellman-Ford algorithm

- Case 2: G contains a negative cycle which is reachable from s



$c=<v_o\ v_1\ \ldots\ v_k>$ is a negative cycle

$$\sum_{i=1}^{k} w(v_{i-1}, v_i) < 0$$

Proof by Contradiction: suppose the algorithm returns a solution

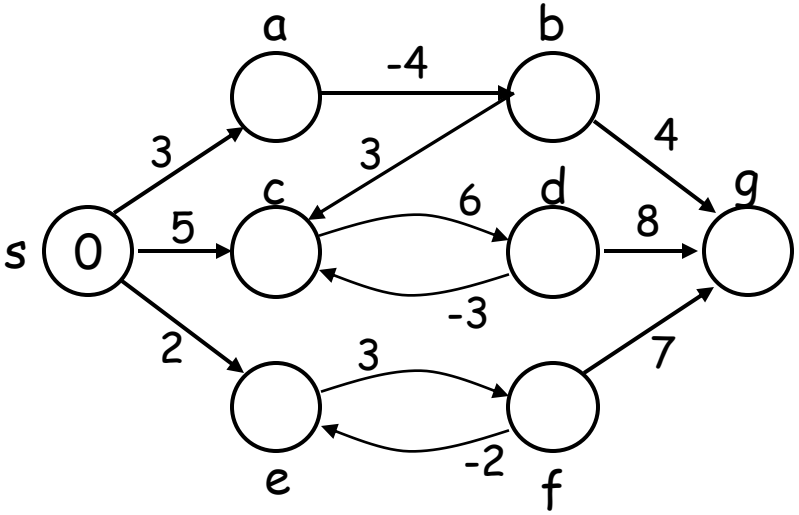
After relaxing (v_{i-1}, v_i): $dist[v_i] \le dist[v_{i-1}] + w(v_{i-1}, v_i)$

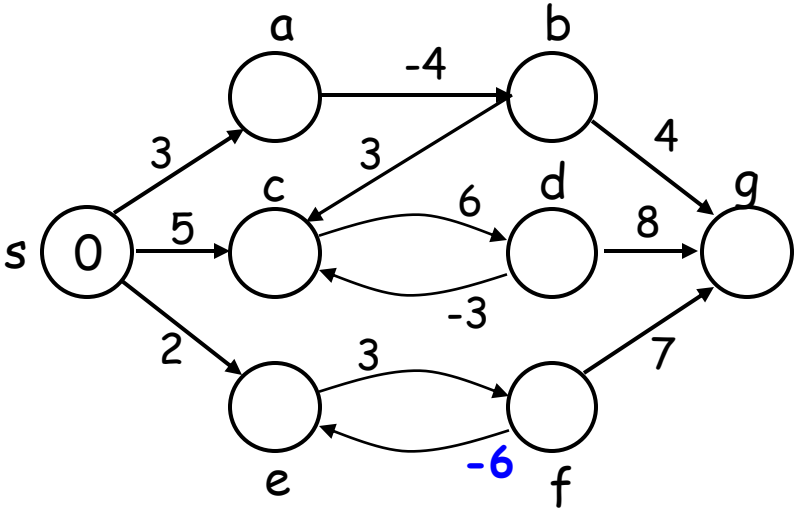
$$\Rightarrow \sum_{i=1}^{k} dist[v_i] \le \sum_{i=1}^{k} dist[v_{i-1}] + \sum_{i=1}^{k} w(v_{i-1}, v_i)$$

$$\Rightarrow \sum_{i=1}^{k} w(v_{i-1}, v_i) \ge 0 \quad (\sum_{i=1}^{k} dist[v_i] = \sum_{i=1}^{k} dist[v_{i-1}])$$

Contradiction!

Exercise 1





Minimum Spanning Tree

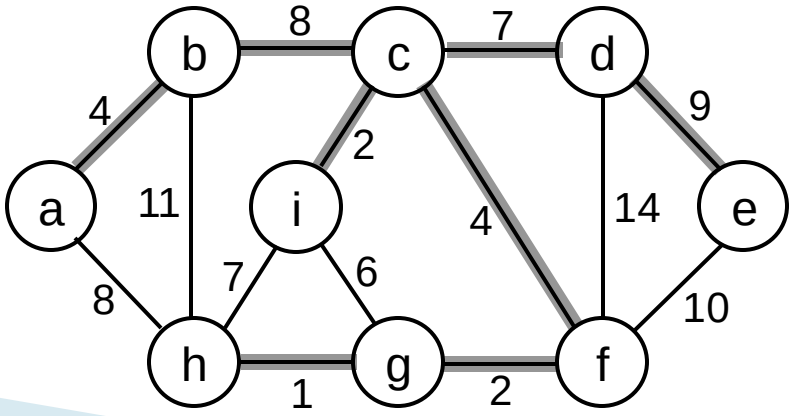
Outline

- What is minimum spanning tree (MST)?
  - Definition and applications
- A generic approach
  - Theoretic proof
  - Prim's algorithm
  - Kruskal's algorithm

Minimum spanning trees

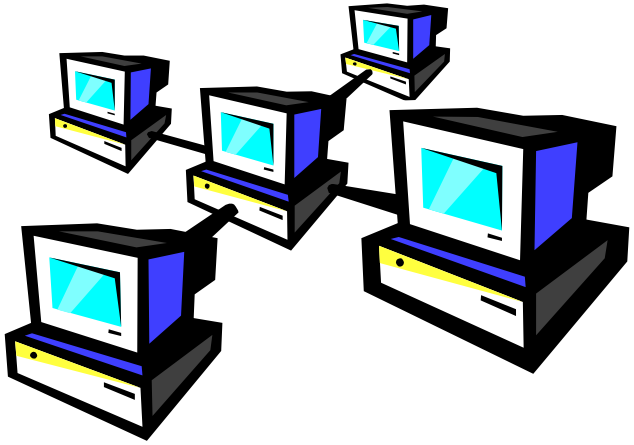
- Spanning tree
  - A tree (i.e., connected, acyclic graph) which contains all the vertices of the graph

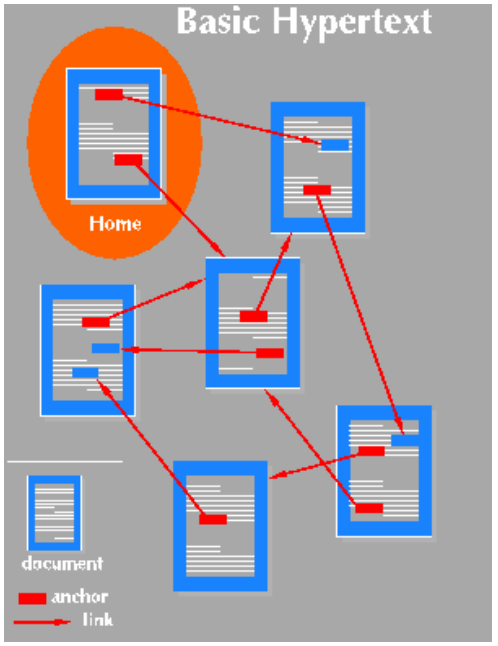
- Minimum spanning tree (MST)
  - Spanning tree with the minimum sum of weights
  - If a graph is not connected, then there is an MST for each connected component of the graph



Applications of MST

- Find the least expensive way to connect a set of houses, cities, terminals, computers, etc.

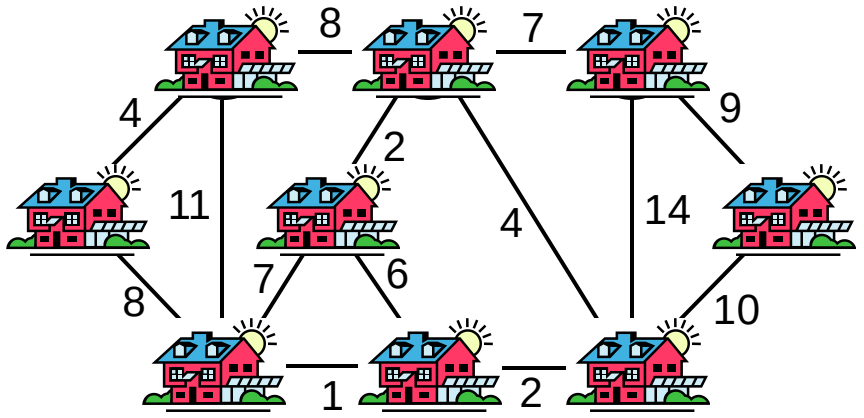




Example

Problem

- A town has a set of houses and a set of roads
- A road connects 2 and only 2 houses
- A road connecting houses u and v has a cost w(u, v)



Goal: Build enough (and no more) roads such that:

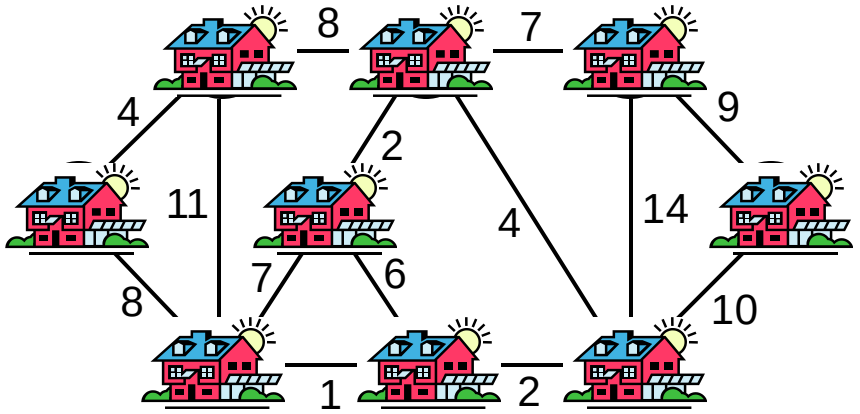
1. Everyone stays connected
   i.e., can reach every house from all other houses
2. Total cost is minimum

Minimum spanning trees (MSTs)

- A connected, undirected graph
  - Vertices = houses
  - Edges = roads
- A weighted w(u, v) on each edge (u, v) $\in$ E

Find $T \subseteq E$ such that:

1. T connects all vertices
2. $w(T) = \Sigma_{(u,v)\in T} w(u, v)$ is minimized

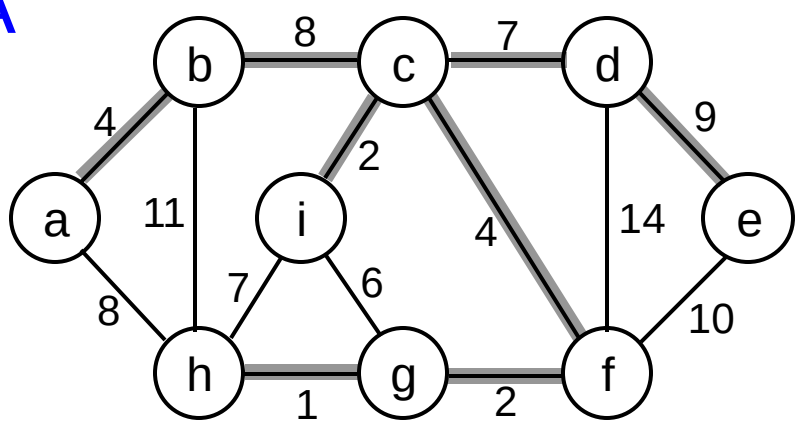


Properties of MST:

(1) MST is **not** unique; (2) MST has no cycles;

Growing an MST: generic approach

- Grow a set A of edges (initially empty)
- Incrementally add edges to A such that they would belong to an MST

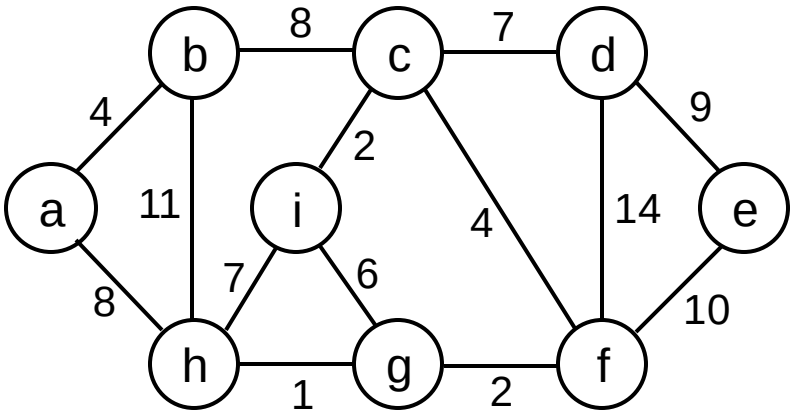


Idea: add only "safe" edges

- An edge (u, v) is **safe** for A, if and only if $A \cup \{(u, v)\}$ is also a subset of **some** MST

Generic MST algorithm

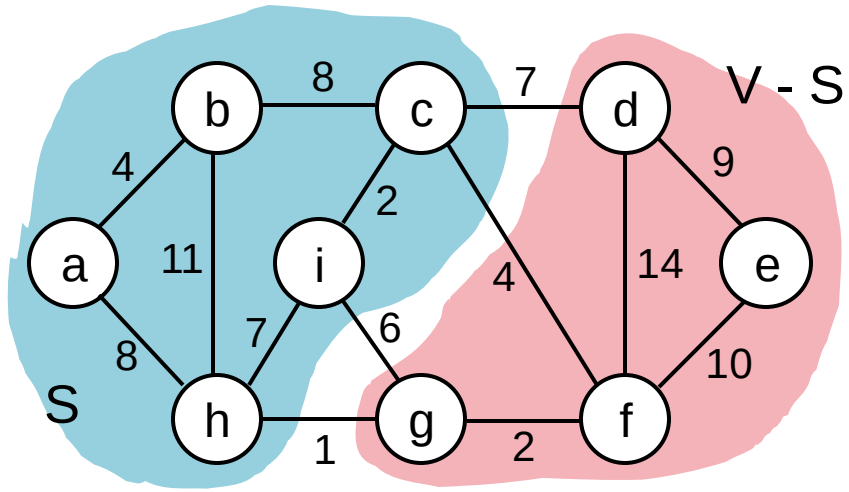
1. $A \leftarrow \varnothing$
2. **while** A is not a spanning tree
3. **do** find an edge (u, v) that is safe for A
4. $A \leftarrow A \cup \{(u, v)\}$
5. **return** A



- How do we find safe edges?

Finding safe edges

- Let's look at edge (h, g)
  - Is it safe for A initially?

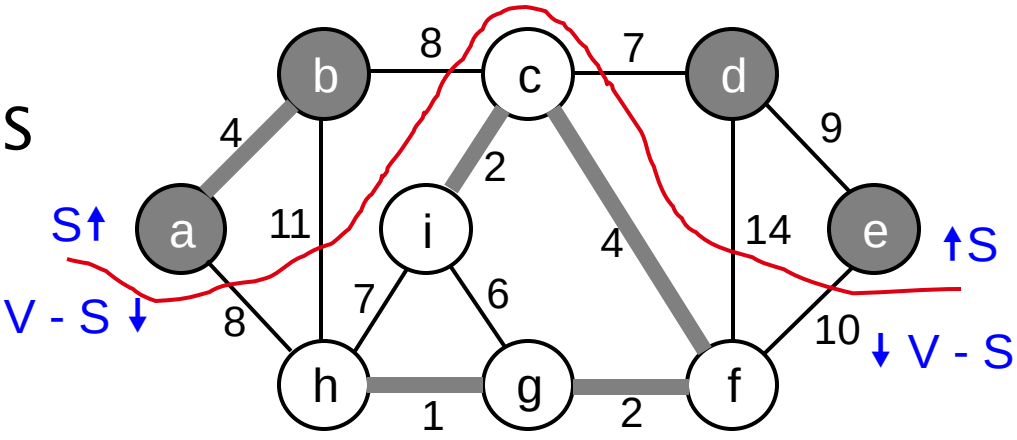


- Yes. Why?
  - Let $S \subset V$ be any set of vertices that includes h but not g (so that g is in V – S)
  - In any MST, there has to be one edge (at least) that connects S with V – S
  - Why not choose the edge with minimum weight (h, g)?

Definitions

- A **cut** (S, V – S)
is a partition of vertices
into two disjoint sets S and V – S

- An edge **crosses** the cut
(S, V – S) if one endpoint is in S
and the other in V – S



- A cut **respects** a set A of edges ⇔ no edge in A crosses the cut

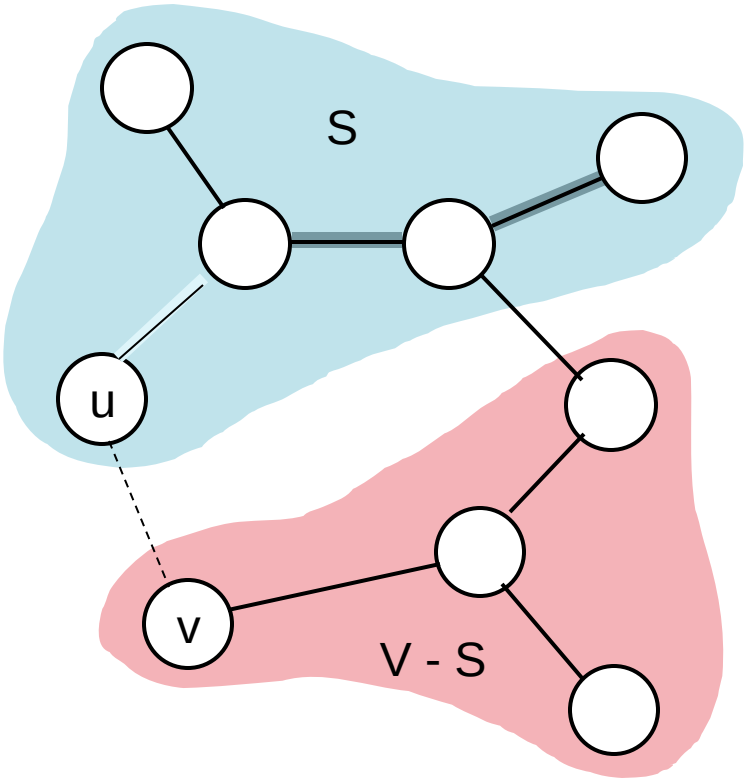
- An edge is a **light edge** crossing a cut
⇔ its weight is minimum over all edges crossing the cut
  - Note that for a given cut, there can be > 1 light edges crossing it

Theorem

- Let *A* be a subset of some MST, (S, V – S) be a **cut** that respects A, and (u, v) be a **light edge** crossing (S, V–S). Then (u, v) is safe for A.

Proof:

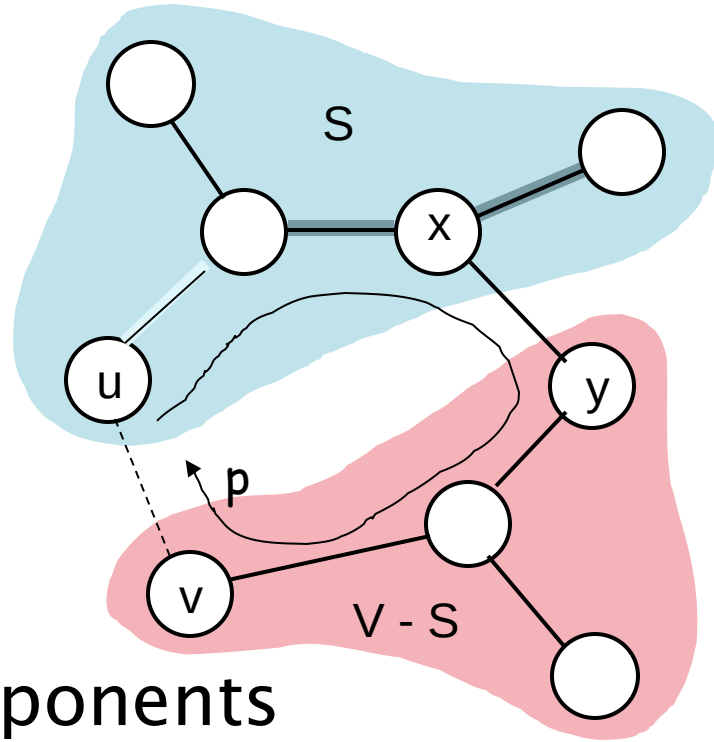
- Let T be an MST that includes A
  - edges in A are shaded
- Case1: If T includes (u,v), then it would be safe for A
- Case2: Suppose T does not include the edge (u, v)
  - **Idea**: construct another MST T' that includes A + {(u, v)}



Theorem: proof

- T contains a unique path p between u and v
- Path p must cross the cut (S, V – S) at least once: let (x, y) be that edge
- Let's remove (x, y) ⇒ breaks T into two components
- Adding (u, v) reconnects the components

$$T' = T - \{(x, y)\} + \{(u, v)\}$$



Theorem: proof

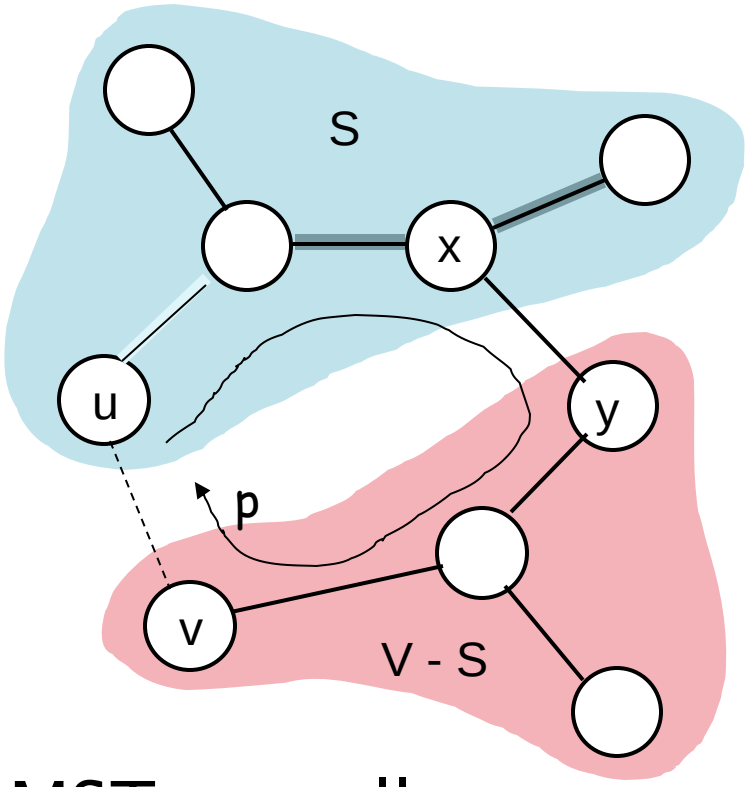
$T' = T - \{(x, y)\} + \{(u, v)\}$

Have to show that T ' is an MST:

- (u, v) is a light edge
  $\Rightarrow w(u, v) \leq w(x, y)$

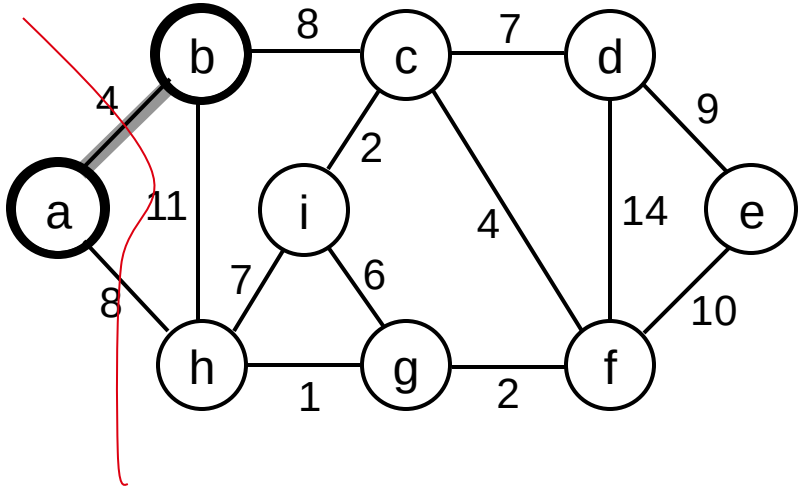
- $w(T') = w(T) - w(x, y) + w(u, v)$
  $\leq w(T)$

- Since T is an MST,
  $w(T) \leq w(T') \Rightarrow$ T ' must be an MST as well



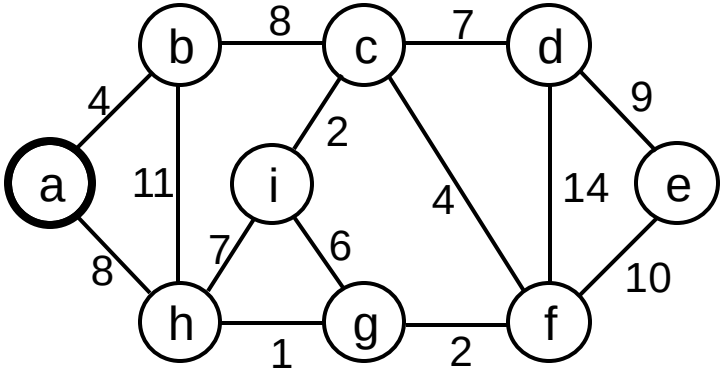
Prim's algorithm

- The edges in set A always <u>form a single tree</u>
- Starts from an arbitrary "root": $V_A = \{a\}$
- At each step:
  - Find a light edge crossing $(V_A, V - V_A)$
  - Add this edge to A
  - Repeat until the tree spans all vertices

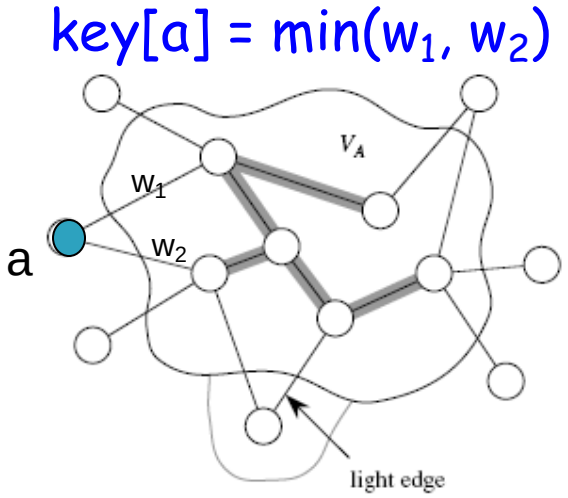


How to find light edges quickly?

- Use a priority queue Q to include vertices not in the tree, i.e., $(V - V_A)$
  - $V_A = \{a\}$, $Q = \{b, c, d, e, f, g, h, i\}$

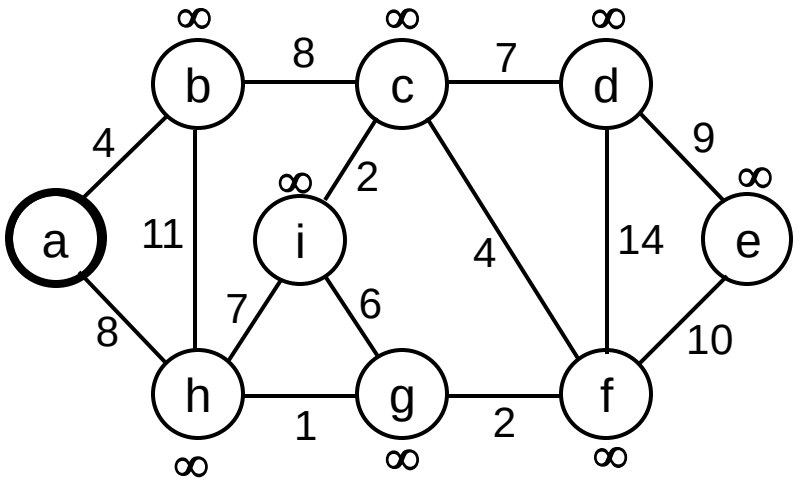


- Associate a key to each vertex v in Q:
  - key[v] = minimum weight of any edge (u, v) connecting v to V_A
  - key[v] = ∞, if v is not adjacent to any vertices in V_A



- After adding a new vertex to V_A, update the weights of all vertices <u>adjacent to it</u>
  - E.g., after adding a, k[b]=4 and k[h]=8

Example

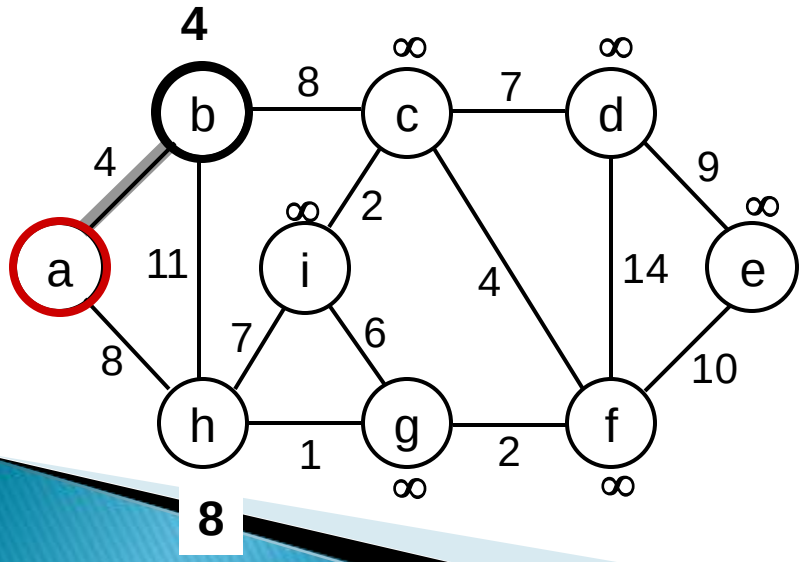


0 ∞ ∞ ∞ ∞ ∞ ∞ ∞ ∞

Q = {a, b, c, d, e, f, g, h, i}

$V_A = \varnothing$

Extract-MIN(Q) ⇒ a



key [b] = 4 π [b] = a

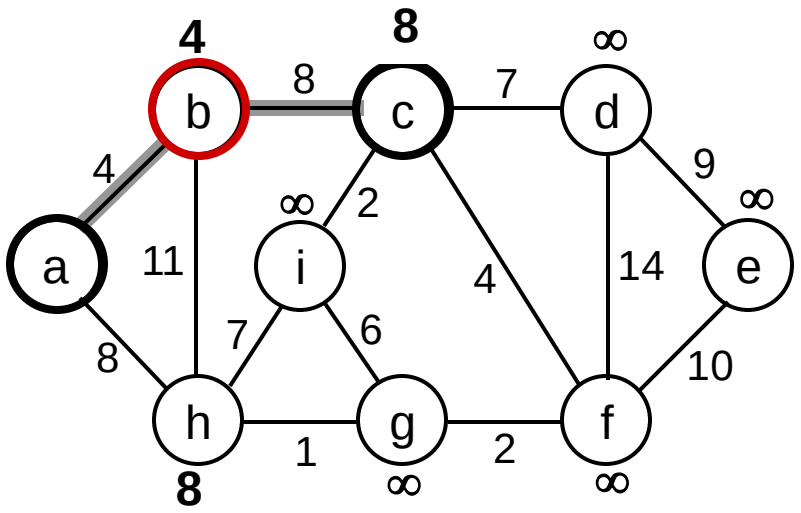
key [h] = 8 π [h] = a

4 ∞ ∞ ∞ ∞ ∞ **8** ∞

Q = {b, c, d, e, f, g, h, i} V_A = {a}

Extract-MIN(Q) ⇒ b

Example



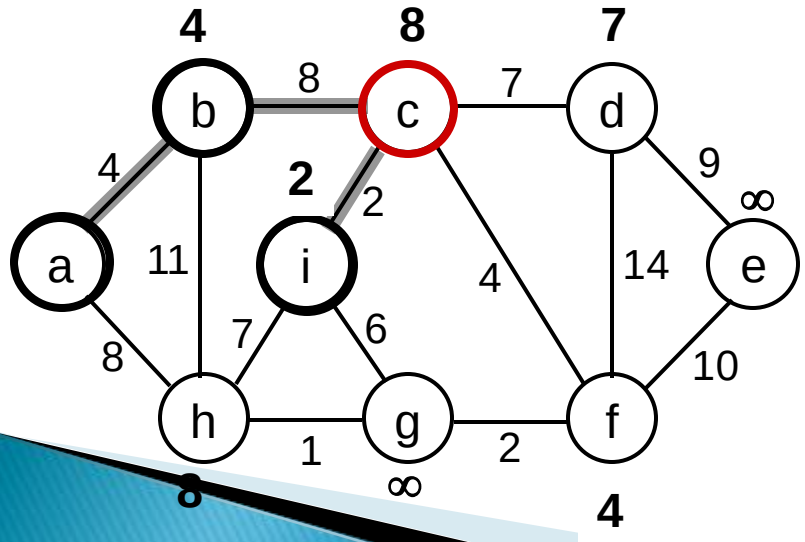
key [c] = 8 π [c] = b

key [h] = 8 π [h] = a unchanged

8 ∞ ∞ ∞ ∞ **8** ∞

Q = {c, d, e, f, g, h, i} V_A = {a, b}

Extract-MIN(Q) ⇒ c



key [d] = 7 π [d] = c

key [f] = 4 π [f] = c

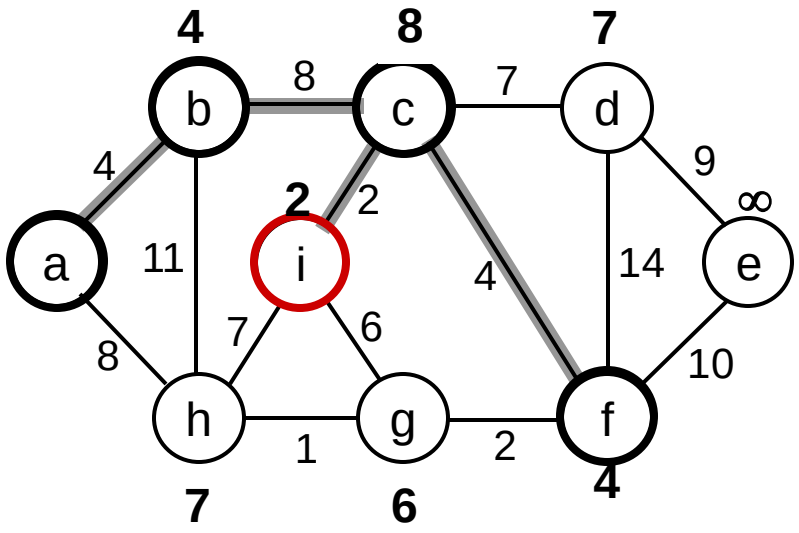
key [i] = 2 π [i] = c

7 ∞ **4** ∞ **8** **2**

Q = {d, e, f, g, h, i} V_A = {a, b, c}

Extract-MIN(Q) ⇒ i

Example



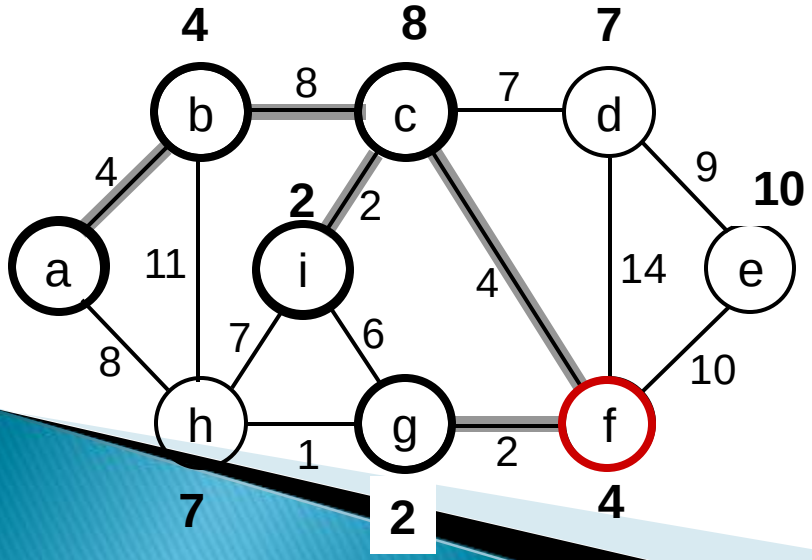
key [h] = 7 π [h] = i

key [g] = 6 π [g] = i

7 ∞ 4 6 7

Q = {d, e, f, g, h} V_A = {a, b, c, i}

Extract-MIN(Q) ⇒ f



key [g] = 2 π [g] = f

key [d] = 7 π [d] = c unchanged

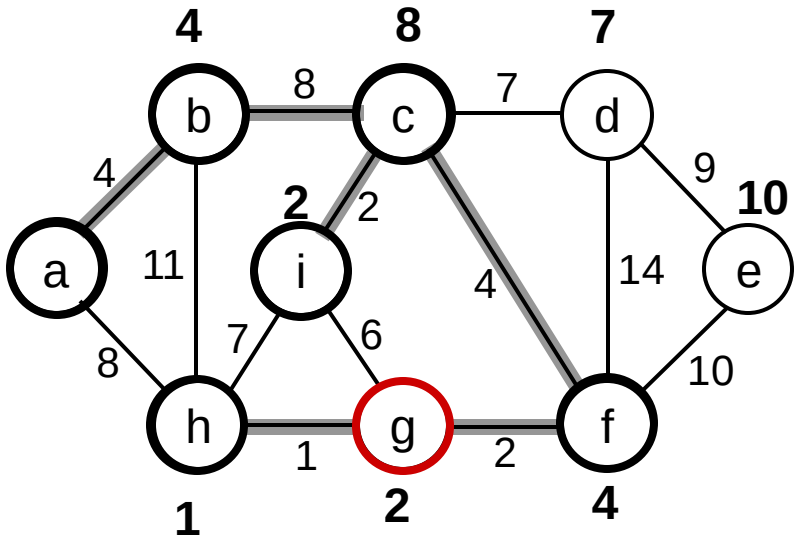
key [e] = 10 π [e] = f

7 10 2 7

Q = {d, e, g, h} V_A = {a, b, c, i, f}

Extract-MIN(Q) ⇒ g

Example

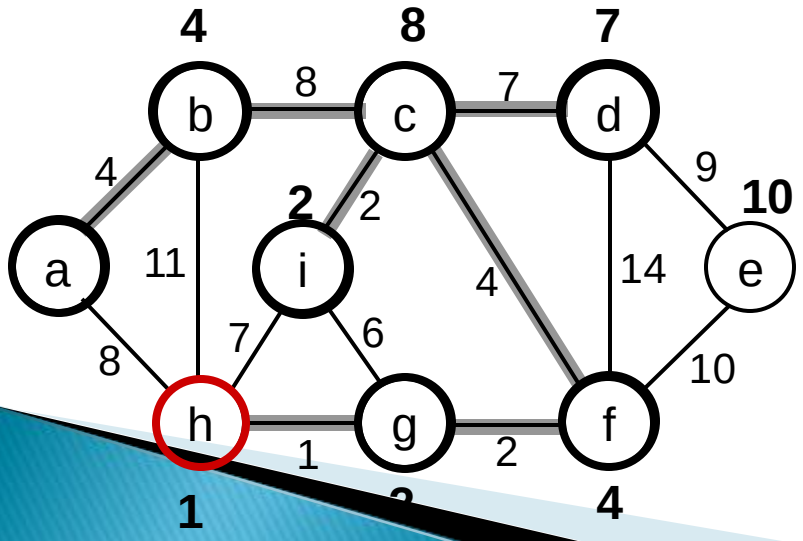


key [h] = 1 π [h] = g

7 10 1

Q = {d, e, h} V_A = {a, b, c, i, f, g}

Extract-MIN(Q) $\Rightarrow$ h

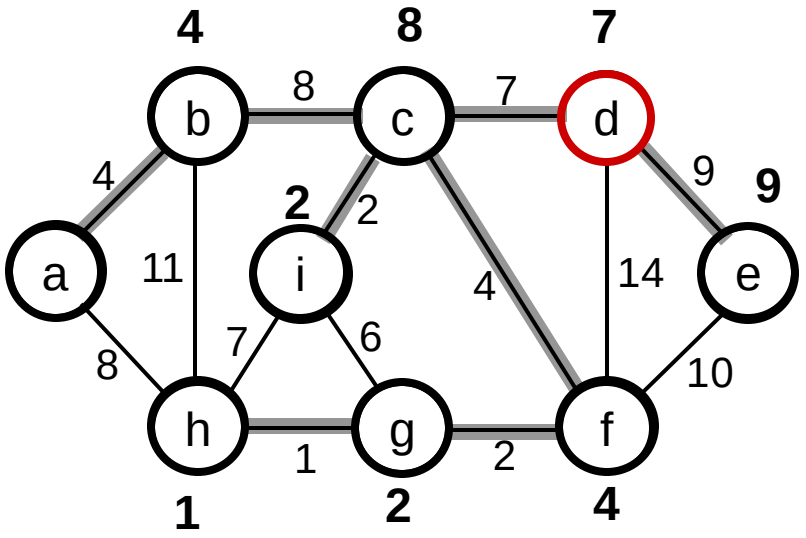


7 10

Q = {d, e} V_A = {a, b, c, i, f, g, h}

Extract-MIN(Q) $\Rightarrow$ d

Example



key [e] = 9 π [e] = d

9

Q = {e} V_A = {a, b, c, i, f, g, h, d}

Extract-MIN(Q) $\Rightarrow$ e

Q = $\varnothing$ V_A = {a, b, c, i, f, g, h, d, e}

Prim(V, E, w, r)

```
1.  Q ← ∅
2.  for each u ∈ V
3.      do key[u] ← ∞
4.         π[u] ← NIL
5.         INSERT(Q, u)
6.  DECREASE-KEY(Q, r, 0)        ▶ key[r] ← 0
7.  while Q ≠ ∅
8.      do u ← EXTRACT-MIN(Q)
9.         for each v ∈ Adj[u]
10.           do if v ∈ Q and w(u, v) < key[v]
11.              then π[v] ← u
12.                   DECREASE-KEY(Q, v, w(u, v))
```

O(V) if Q is implemented as a min-heap

Total time: O(VlgV + ElgV) = O(ElgV)

O(lgV)

Executed |V| times

Takes O(lgV)

Min-heap operations: O(VlgV)

Executed O(E) times

Constant

Takes O(lgV)

O(ElgV)

Prim(V, E, w, r)

```
1.  Q ← ∅
2.  for each u ∈ V
3.       do key[u] ← ∞
4.          π[u] ← NIL
5.          INSERT(Q, u)
6.  DECREASE-KEY(Q, r, 0)        ▶ key[r] ← 0
7.  while Q ≠ ∅
8.       do u ← EXTRACT-MIN(Q)
9.          for (j=0; j<|V|; j++)
10.            if (A[u][j]=1)
11.              if v ∈ Q and w(u, v) < key[v]
12.                then π[v] ← u
13.                     DECREASE-KEY(Q, v, w(u, v))
```

Total time: $O(V \lg V + E \lg V + V^2) = O(E \lg V + V^2)$

$O(V)$ if Q is implemented as a min-heap

$O(\lg V)$

Executed $|V|$ times

Min-heap operations: $O(V \lg V)$

Takes $O(\lg V)$

Executed $O(V^2)$ times total

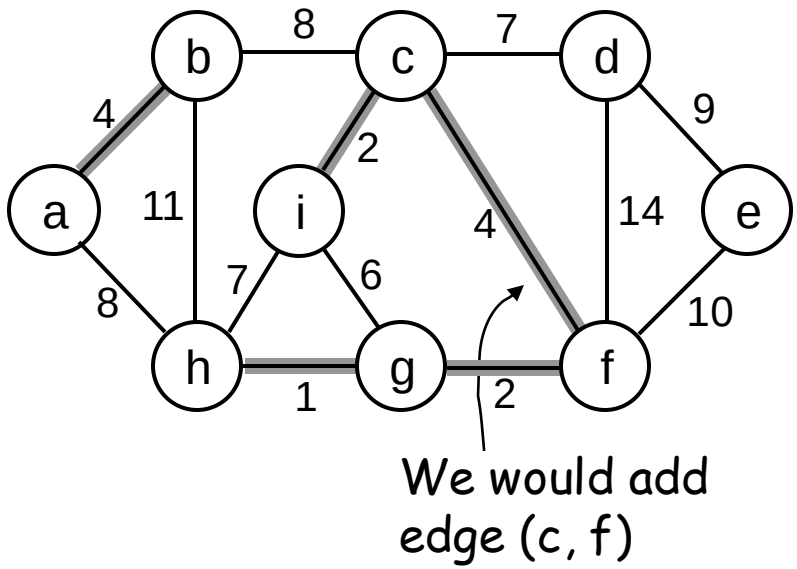
Constant

Takes $O(\lg V)$

$O(E \lg V)$

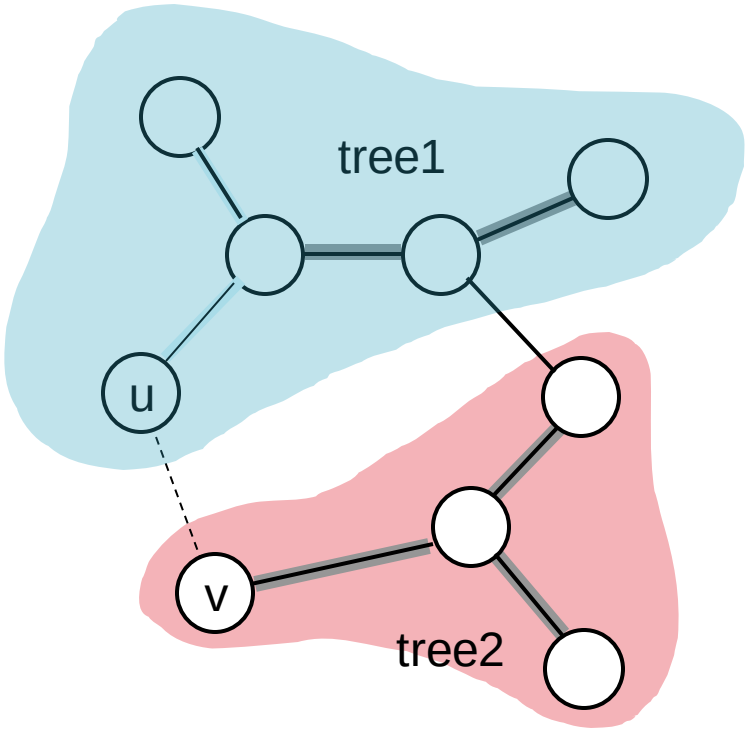
Kruskal's algorithm

- Start with each vertex being its own component
- Repeatedly merge two components into one by choosing the **light** edge that connects them
- Which components to consider at each iteration?
  - Scan the set of edges in monotonically increasing order by weight

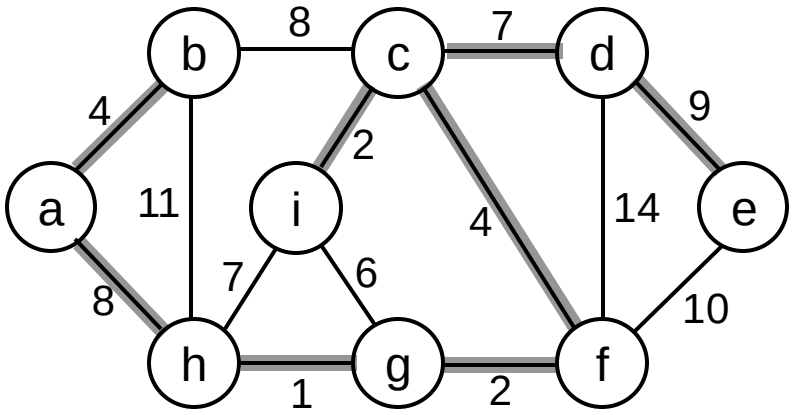


Kruskal's algorithm

- How is it different from Prim's algorithm?
  - Prim's algorithm grows one tree all the time
  - Kruskal's algorithm grows multiple trees (i.e., a forest) at the same time
  - Trees are merged together using **safe** edges



Example



1: (h, g)
2: (c, i), (g, f)
4: (a, b), (c, f)
6: (i, g)
7: (c, d), (i, h)
8: (a, h), (b, c)
9: (d, e)
10: (e, f)
11: (b, h)
14: (d, f)

{a}, {b}, {c}, {d}, {e}, {f}, {g}, {h}, {i}

1.	Add (h, g)	{g, h}, {a}, {b}, {c}, {d}, {e}, {f}, {i}
2.	Add (c, i)	{g, h}, {c, i}, {a}, {b}, {d}, {e}, {f}
3.	Add (g, f)	{g, h, f}, {c, i}, {a}, {b}, {d}, {e}
4.	Add (a, b)	{g, h, f}, {c, i}, {a, b}, {d}, {e}
5.	Add (c, f)	{g, h, f, c, i}, {a, b}, {d}, {e}
6.	Ignore (i, g)	{g, h, f, c, i}, {a, b}, {d}, {e}
7.	Add (c, d)	{g, h, f, c, i, d}, {a, b}, {e}
8.	Ignore (i, h)	{g, h, f, c, i, d}, {a, b}, {e}
9.	Add (a, h)	{g, h, f, c, i, d, a, b}, {e}
10.	Ignore (b, c)	{g, h, f, c, i, d, a, b}, {e}
11.	Add (d, e)	{g, h, f, c, i, d, a, b, e}
12.	Ignore (e, f)	{g, h, f, c, i, d, a, b, e}
13.	Ignore (b, h)	{g, h, f, c, i, d, a, b, e}
14.	Ignore (d, f)	{g, h, f, c, i, d, a, b, e}

Algorithm 1: a naive method

Assume vertices are 1, 2, …, n, and E >= V

```
1.  Sort all the edges                                  <-- O(ElogE)
2.  for each v ∈ V                                      } O(V)
3.      map.add(v, {v})                                 }
4.  for each edge (u, v) ∈ E
5.      setU = map.get(u), setV = map.get(v)            } O(1)
6.      isConnected = false                             }
7.      for each vertex w ∈ setU                        } O(setU.size)
8.          if w == v                                   }
9.              isConnected = true                      }
10.             break                                   }
11.     if isConnected == false                         } O(setV.size)
12.         setU = setU ∪ setV                          }
13.         map.add(u, setU), map.add(v, setU)          }
14.         R = R ∪ {(u, v)}                            }
15. Output R
```

O(VE)

Can we do better?

Algorithm 2: using labels

- Using labels
  - A label means a connected component
  - Assign a unique label to each vertex initially
  - When merging two connected components, we always change the labels of vertices in the small component to the label of the large component
    - The cost of changing labels is smaller

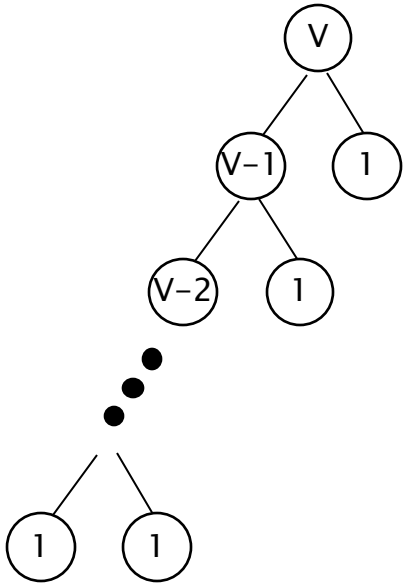
Algorithm 2: using labels

Assume vertices are 1, 2, …, n, and E >= V

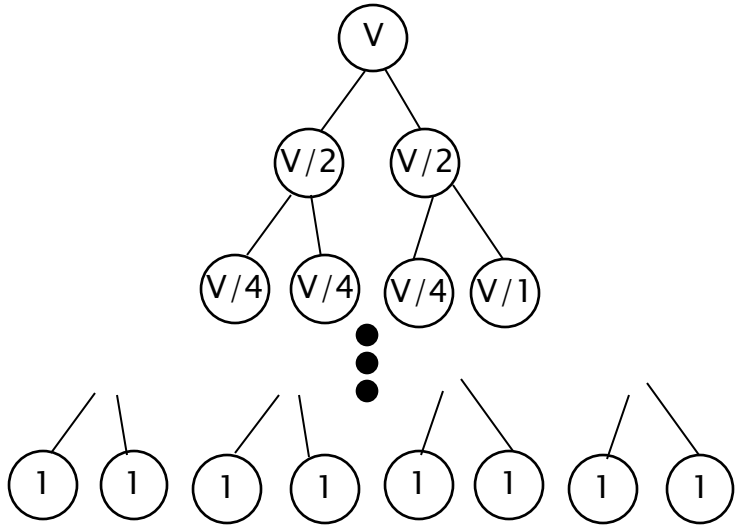
```
1.      Sort all the edges                                  ← O(ElogE)
2.      for each v ∈ V                                      }
3.          label[v] = v                                    } O(V)
4.          setArray[v] = {v}                               }
5.      for each edge (u, v) ∈ E                            }
6.          uL = label[u], vL = label[v]                    } O(E)
7.          if uL == vL  continue                           }
8.          R.add((u, v))                                   }
9.        if setArray[uL].size >= setArray[vL].size         }
10.         for each vertex w ∈ setArray[vL]                } O(setArray[vL].size)
11.             label[w] = uL                               }
12.             setArray[uL].add(w)                         }
13.       else                                              }
14.           for each vertex w ∈ setArray[uL]              } O(setArray[uL].size)
15.               label[w] = vL                             }
16.               setArray[vL].add(w)                       }
17.     Output R                                            } ???
```

What's the total times of changing the labels for all the vertices?

- Extreme case 1
  - Height: V−1
  - Change O(V) times



- Extreme case 2
  - Height: lgV
  - Change O(VlgV) times



Consider a specific vertex v: if v's label is changed, then the updated set setArray[vL] will be at least twice larger than the original set setArray[vL]. Hence, the number of times for changing labels for v is at most O(lgV).

Algorithm 2: using labels

Assume vertices are 1, 2, …, n, and E >= V

```
1.    Sort all the edges                                  <-- O(ElogE)
2.    for each v ∈ V                                      } O(V)
3.        label[v] = v                                    }
4.        setArray[v] = {v}                               }
5.    for each edge (u, v) ∈ E                            } O(E)
6.        uL = label[u], vL = label[v]                    }
7.        if uL == vL  continue                           }
8.        R.add((u, v))                                   }
9.      if setArray[uL].size >= setArray[vL].size         } O(VlogV)
10.       for each vertex w ∈ setArray[vL]                }
11.           label[w] = uL                               }
12.           setArray[uL].add(w)                         }
13.     else                                              }
14.         for each vertex w ∈ setArray[uL]              }
15.             label[w] = vL                             }
16.             setArray[vL].add(w)                       }
17.   Output R                                            } O(ElgE)
```

Exercises

- Given the following graph, find its MST using Prim's algorithm and Kruskal's algorithm respectively

